

5.11.8 Programming In C: GATE CSE 2012 | Question: 49



Consider the following C code segment.

```
int a, b, c = 0;
void prtFun(void);
main()
{
    static int a = 1; /* Line 1 */
    prtFun();
    a += 1;
    prtFun();
    printf("\n %d %d ", a, b);
}

void prtFun(void)
{
    static int a = 2; /* Line 2 */
    int b = 1;
    a += ++b;
    printf("\n %d %d ", a, b);
}
```

What output will be generated by the given code segment if:

Line 1 is replaced by **auto** `int a = 1;`

Line 2 is replaced by **register** `int a = 2;`

- | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|--|---|---|---|---|---|---|--|---|---|---|---|---|---|--|---|---|---|---|---|---|--|---|---|---|---|---|---|
| A. <table border="1"><tr><td>3</td><td>1</td></tr><tr><td>4</td><td>1</td></tr><tr><td>4</td><td>2</td></tr></table> | 3 | 1 | 4 | 1 | 4 | 2 | B. <table border="1"><tr><td>4</td><td>2</td></tr><tr><td>6</td><td>1</td></tr><tr><td>6</td><td>1</td></tr></table> | 4 | 2 | 6 | 1 | 6 | 1 | C. <table border="1"><tr><td>4</td><td>2</td></tr><tr><td>6</td><td>2</td></tr><tr><td>2</td><td>0</td></tr></table> | 4 | 2 | 6 | 2 | 2 | 0 | D. <table border="1"><tr><td>4</td><td>2</td></tr><tr><td>4</td><td>2</td></tr><tr><td>2</td><td>0</td></tr></table> | 4 | 2 | 4 | 2 | 2 | 0 |
| 3 | 1 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 4 | 1 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 4 | 2 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 4 | 2 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 6 | 1 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 6 | 1 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 4 | 2 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 6 | 2 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 2 | 0 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 4 | 2 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 4 | 2 | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 2 | 0 | | | | | | | | | | | | | | | | | | | | | | | | | | |

normal gatecse-2012 programming-in-c programming

Answer key

5.11.9 Programming In C: GATE CSE 2014 | Set 2 | Question: 11



Suppose n and p are unsigned int variables in a C program. We wish to set p to nC_3 . If n is large, which one of the following statements is most likely to set p correctly?

- | | |
|---|---|
| A. $p = n * (n - 1) * (n - 2) / 6;$ | B. $p = n * (n - 1) / 2 * (n - 2) / 3;$ |
| C. $p = n * (n - 1) / 3 * (n - 2) / 2;$ | D. $p = n * (n - 1) * (n - 2) / 6.0;$ |

gatecse-2014-set2 programming programming-in-c normal

Answer key

5.11.10 Programming In C: GATE CSE 2014 | Set 2 | Question: 42



Consider the C function given below.

```
int f(int j)
{
    static int i = 50;
    int k;
    if (i == j)
    {
        printf("something");
        k = f(i);
        return 0;
    }
    else return 0;
}
```

Which one of the following is **TRUE**?

- | |
|---|
| A. The function returns 0 for all values of j . |
| B. The function prints the string something for all values of j . |
| C. The function returns 0 when $j = 50$. |
| D. The function will exhaust the runtime stack or run into an infinite loop when $j = 50$. |

gatecse-2014-set2 programming programming-in-c

Answer key

5.11.11 Programming In C: GATE CSE 2015 | Set 1 | Question: 11



The output of the following C program is _____.

```
void f1 (int a, int b) {  
    int c;  
    c = a; a = b;  
    b = c;  
}  
void f2 (int *a, int *b) {  
    int c;  
    c = *a; *a = *b; *b = c;  
}  
int main () {  
    int a = 4, b = 5, c = 6;  
    f1 (a, b);  
    f2 (&b, &c);  
    printf ("%d", c - a - b);  
}
```

gatecse-2015-set1 programming programming-in-c easy numerical-answers

Answer key

5.11.12 Programming In C: GATE CSE 2016 | Set 1 | Question: 12



Consider the following "C" program.

```
void f(int, short);  
void main()  
{  
    int i = 100;  
    short s = 12;  
    short *p = &s;  
    _____; // call to f()  
}
```

Which one of the following expressions, when placed in the blank above, will NOT result in a type checking error?

- A. $f(s, *s)$ B. $i = f(i, s)$ C. $f(i, *s)$ D. $f(i, *p)$

gatecse-2016-set1 programming-in-c easy

Answer key

5.11.13 Programming In C: GATE CSE 2016 | Set 1 | Question: 34



The following function computes the maximum value contained in an integer array $P[]$ of size n ($n \geq 1$).

```
int max (int *p, int n) {  
    int a = 0, b = n - 1;  
  
    while (_____) {  
        if (p[a] <= p[b]) {a = a + 1;}  
        else {b = b - 1;}  
    }  
    return p[a];  
}
```

The missing loop condition is:

- A. $a \neq n$ B. $b \neq 0$
C. $b > (a + 1)$ D. $b \neq a$

gatecse-2016-set1 programming-in-c normal

Answer key

5.11.14 Programming In C: GATE CSE 2017 | Set 1 | Question: 53



Consider the following C program.

```
#include <stdio.h>
```

```
#include<string.h>

void printlength(char *s, char t) {
    unsigned int c=0;
    int len = ((strlen(s) > strlen(t)) ? strlen(s) : strlen(t));
    printf("%d\n", len);
}

void main() {
    char *x = "abc";
    char *y = "defgh";
    printlength(x,y);
}
```

Recall that *strlen* is defined in *string.h* as returning a value of type *size_t*, which is an unsigned int. The output of the program is _____.

gatecse-2017-set1 programming programming-in-c normal numerical-answers

Answer key

5.11.15 Programming In C: GATE CSE 2017 | Set 1 | Question: 55



The output of executing the following C program is _____.

```
#include<stdio.h>

int total(int v) {
    static int count = 0;
    while(v) {
        count += v&1;
        v >>= 1;
    }
    return count;
}

void main() {
    static int x=0;
    int i=5;
    for(; i>0; i--) {
        x = x + total(i);
    }
    printf("%d\n", x);
}
```

gatecse-2017-set1 programming programming-in-c normal numerical-answers

Answer key

5.11.16 Programming In C: GATE CSE 2017 | Set 2 | Question: 2



Match the following:

P. static char var ;	i. Sequence of memory locations to store addresses
Q. m = malloc(10); m=NULL ;	ii. A variable located in data section of memory
R. char *ptr[10] ;	iii. Request to allocate a CPU register to store data
S. register int varl;	iv. A lost memory which cannot be freed

- A. P-ii; Q-iv; R-i; S-iii
C. P-ii; Q-iv; R-iii; S-i

- B. P-ii; Q-i; R-iv; S-iii
D. P-iii; Q-iv; R-i; S-ii

gatecse-2017-set2 programming programming-in-c match-the-following

Answer key

5.11.17 Programming In C: GATE CSE 2017 | Set 2 | Question: 54



Consider the following C program.

```
#include<stdio.h>
int main () {
    int m=10;
    int n, n1;
    n=++m;
```

```

n1=m++;
n--;
--n1;
n-=n1;
printf("%d", n);
return 0;
}

```

The output of the program is _____

gatecse-2017-set2 programming-in-c numerical-answers easy

Answer key

5.11.18 Programming In C: GATE CSE 2018 | Question: 32



Consider the following C code. Assume that unsigned long int type length is 64 bits.

```

unsigned long int fun(unsigned long int n) {
    unsigned long int i, j=0, sum=0;
    for (i=n; i>1; i=i/2) i++;
    for (; j>1; j=j/2) sum++;
    return sum;
}

```

The value returned when we call fun with the input 2^{40} is:

- A. 4 B. 5 C. 6 D. 40

gatecse-2018 programming-in-c normal programming two-marks

Answer key

5.11.19 Programming In C: GATE CSE 2019 | Question: 27



Consider the following C program:

```

#include <stdio.h>
int r() {
    static int num=7;
    return num--;
}
int main() {
    for (r();r();r())
        printf("%d",r());
    return 0;
}

```

Which one of the following values will be displayed on execution of the programs?

- A. 41 B. 52 C. 63 D. 630

gatecse-2019 programming-in-c programming two-marks

Answer key

5.11.20 Programming In C: GATE CSE 2019 | Question: 52



Consider the following C program:

```

#include <stdio.h>
int main() {
    float sum = 0.0, j=1.0, i=2.0;
    while (i/j > 0.0625) {
        j=j+i;
        sum=sum+i/j;
        printf("%f\n", sum);
    }
    return 0;
}

```

The number of times the variable sum will be printed, when the above program is executed, is _____

gatecse-2019 numerical-answers programming-in-c programming two-marks

Answer key

5.11.21 Programming In C: GATE CSE 2019 | Question: 53



Consider the following C program:

```
#include <stdio.h>
int main()
{
    int a[] = {2, 4, 6, 8, 10};
    int i, sum=0, *b=a+4;
    for (i=0; i<5; i++)
        sum=sum+(*b-i)-*(b-i);
    printf("%d\n", sum);
    return 0;
}
```

The output of the above C program is _____

gatecse-2019 numerical-answers programming-in-c programming two-marks

Answer key

5.11.22 Programming In C: GATE CSE 2021 | Set 1 | Question: 37



Consider the following ANSI C program.

```
#include <stdio.h>
int main()
{
    int i, j, count;
    count=0;
    i=0;
    for (j=-3; j<=3; j++)
    {
        if ((j >= 0) && (i++))
            count = count + j;
    }
    count = count + i;
    printf("%d", count);
    return 0;
}
```

Which one of the following options is correct?

- A. The program will not compile successfully
- B. The program will compile successfully and output 10 when executed
- C. The program will compile successfully and output 8 when executed
- D. The program will compile successfully and output 13 when executed

gatecse-2021-set1 programming-in-c two-marks

Answer key

5.11.23 Programming In C: GATE CSE 2024 | Set 1 | Question: 38



Consider the following C function definition.

```
int f (int x, int y){
    for (int i=0 ; i<y ; i++ ) {
        x= x + x + y;
    }
    return x;
}
```

Which of the following statements is/are TRUE about the above function?

- A. If the inputs are $x = 20, y = 10$, then the return value is greater than 2^{20}
- B. If the inputs are $x = 20, y = 20$, then the return value is greater than 2^{20}
- C. If the inputs are $x = 20, y = 10$, then the return value is less than 2^{10}
- D. If the inputs are $x = 10, y = 20$, then the return value is greater than 2^{20}

gatecse-2024-set1 multiple-selects programming programming-in-c two-marks

Answer key

5.11.24 Programming In C: GATE CSE 2026 | Set 2 | Question: 9



Consider the following three ANSI-C programs, P1, P2, and P3.

P1	P2	P3
<pre>#include <stdio.h> int a=5; int main(){ int a=7; return(0); }</pre>	<pre>#include <stdio.h> int main(){ int a=5; int a=7; return(0); }</pre>	<pre>#include <stdio.h> int main(){ int a=5; float a=7; return(0); }</pre>

Which one of the following statements is true?

- A. Only P1 will compile without any error
- B. Only P2 will compile without any error
- C. Only P3 will compile without any error
- D. All three programs P1, P2, and P3 will compile without any error

gatecse-2026-set2 programming-in-c one-mark

Answer key

5.11.25 Programming In C: GATE IT 2004 | Question: 59



What is the output of the following program?

```
#include<stdio.h>
int funcf (int x);
int funcg (int y);
main ()
{
    int x = 5, y = 10, count;
    for (count = 1; count <= 2; ++count) {
        y += funcf(x) + funcg(x);
        printf ("%d", y);
    }
}
funcf (int x) {
    int y;
    y = funcg(x);
    return (y);
}
funcg (int x) {
    static int y = 10;
    y += 1;
    return (y + x);
}
```

A. 43 80

B. 42 74

C. 33 37

D. 32 32

gateit-2004 programming programming-in-c normal

Answer key

5.11.26 Programming In C: GATE IT 2004 | Question: 60



Choose the correct option to fill the ?1 and ?2 so that the program prints an input string in reverse order. Assume that the input string is terminated by a new line character.

```
#include <stdio.h>
void wrt_it (void);
int main (void)
{
    printf("Enter Text");
```

```

printf("\n");
wrt_it();
printf("\n");
return 0;
}

void wrt_it(void)
{
    int c;
    if (?1)
        wrt_it();
    ?2
}

```

- A. ?1 is `getchar() != '\n'`
 ?2 is `getchar(c);`
- B. ?1 is `(c = getchar()); != '\n'`
 ?2 is `getchar(c);`
- C. ?1 is `c != '\n'`
 ?2 is `putchar(c);`
- D. ?1 is `(c = getchar()) != '\n'`
 ?2 is `putchar(c);`

gateit-2004 programming programming-in-c normal

Answer key

5.11.27 Programming In C: GATE IT 2005 | Question: 58



Let a be an array containing n integers in increasing order. The following algorithm determines whether there are two distinct numbers in the array whose difference is a specified number $S > 0$.

```

i = 0; j = 1;
while (j < n) {
    if (E) j++;
    else if (a[j] - a[i] == S) break;
    else i++;
}
if (j < n) printf("yes") else printf("no");

```

Choose the correct expression for E.

- A. $a[j] - a[i] > S$
- B. $a[j] - a[i] < S$
- C. $a[i] - a[j] < S$
- D. $a[i] - a[j] > S$

gateit-2005 programming normal programming-in-c

Answer key

5.11.28 Programming In C: GATE IT 2006 | Question: 51



Which one of the choices given below would be printed when the following program is executed?

```

#include <stdio.h>
int a1[] = {6, 7, 8, 18, 34, 67};
int a2[] = {23, 56, 28, 29};
int a3[] = {-12, 27, -31};
int *x[] = {a1, a2, a3};
void print(int *a[])
{
    printf("%d", a[0][2]);
    printf("%d", *a[2]);
    printf("%d", **a[0]);
    printf("%d", *(++a[0]));
    printf("%d\n", a[-1][+1]);
}

main()
{
    print(x);
}

```

- A. 8, -12, 7, 23, 8
- B. 8, 8, 7, 23, 7
- C. -12, -12, 27, -31, 23
- D. -12, -12, 27, -31, 56

gateit-2006 programming programming-in-c normal

Answer key

5.11.29 Programming In C: GATE IT 2007 | Question: 31



Consider the C program given below :

```
#include <stdio.h>
int main()
{
    int sum = 0, maxsum = 0, i, n = 6;
    int a[] = {2, -2, -1, 3, 4, 2};
    for (i = 0; i < n; i++) {
        if (i == 0 || a[i] < 0 || a[i] < a[i - 1]) {
            if (sum > maxsum) maxsum = sum;
            sum = (a[i] > 0) ? a[i] : 0;
        }
        else sum += a[i];
    }
    if (sum > maxsum) maxsum = sum;
    printf("%d\n", maxsum);
}
```

What is the value printed out when this program is executed?

A. 9

B. 8

C. 7

D. 6

gateit-2007 programming programming-in-c normal

Answer key

5.12 Programming Paradigms (2)

5.12.1 Programming Paradigms: GATE CSE 2004 | Question: 1



The goal of structured programming is to:

- A. have well indented programs
- B. be able to infer the flow of control from the compiled code
- C. be able to infer the flow of control from the program text
- D. avoid the use of GOTO statements

gatecse-2004 programming easy programming-paradigms

Answer key

5.12.2 Programming Paradigms: GATE CSE 2004 | Question: 90



Choose the best matching between the programming styles in Group 1 and their characteristics in Group 2.

Group 1	Group 2
P. Functional	1. Common-based, procedural
Q. Logic	2. Imperative, abstract data types
R. Object-oriented	3. Side-effect free, declarative, expression evaluations
S. Imperative	4. Declarative, clausal representation, theorem proving

A. P - 2 Q - 3 R - 4 S - 1
C. P - 3 Q - 4 R - 1 S - 2

B. P - 4 Q - 3 R - 2 S - 1
D. P - 3 Q - 4 R - 2 S - 1

gatecse-2004 programming normal programming-paradigms match-the-following

Answer key

5.13 Recursion (19)

5.13.1 Recursion: GATE CSE 1991 | Question: 01,x



Consider the following recursive definition of *fib*:

```
fib(n) := if n = 0 then 1
        else if n = 1 then 1
```



```
else fib(n-1) + fib(n-2)
```

The number of times fib is called (including the first call) for evaluation of $fib(7)$ is _____.

gate1991 programming recursion normal numerical-answers

Answer key

5.13.2 Recursion: GATE CSE 1994 | Question: 21



Consider the following recursive function:

```
function fib (n:integer):integer
begin
if (n=0) or (n=1) then fib:=1
else fib:=fib(n-1) + fib(n-2)
end;
```

The above function is run on a computer with a stack of 64 bytes. Assuming that only return address and parameter are passed on the stack, and that an integer value and an address takes 2 bytes each, estimate the maximum value of n for which the stack will not overflow. Give reasons for your answer.

gate1994 programming recursion normal descriptive

Answer key

5.13.3 Recursion: GATE CSE 2000 | Question: 16



A recursive program to compute Fibonacci numbers is shown below. Assume you are also given an array $f[0 \dots m]$ with all elements initialized to 0.

```
fib(n) {
if (n > M) error();
if (n == 0) return 1;
if (n == 1) return 1;
if ( ) (1)
return (2)
t = fib(n-1) + fib(n-2);
return t; (3)
}
```

- A. Fill in the boxes with expressions/statements to make $fib()$ store and reuse computed Fibonacci values. Write the box number and the corresponding contents in your answer book.
- B. What is the time complexity of the resulting program when computing $fib(n)$?

gatecse-2000 algorithms normal descriptive recursion

Answer key

5.13.4 Recursion: GATE CSE 2001 | Question: 13



Consider the following C program:

```
void abc(char*s)
{
if(s[0]!='\0')return;
abc(s+1);
abc(s+1);
printf("%c",s[0]);
}

main()
{
abc("123");
}
```

- A. What will be the output of the program?
- B. If $abc(s)$ is called with a null-terminated string s of length n characters (not counting the null ($\backslash 0$) character), how many characters will be printed by $abc(s)$?

gatecse-2001 programming recursion normal descriptive

Answer key

5.13.5 Recursion: GATE CSE 2002 | Question: 11



The following recursive function in C is a solution to the Towers of Hanoi problem.

```
void move(int n, char A, char B, char C) {  
    if (.....) {  
        move.....;  
        printf("Move disk %d from pole %c to pole %c\n", n, A, C);  
        move (.....);  
    }  
}
```

Fill in the dotted parts of the solution.

gatecse-2002 programming recursion descriptive

Answer key

5.13.6 Recursion: GATE CSE 2004 | Question: 31, ISRO2008-40



Consider the following C function:

```
int f(int n)  
{  
    static int i = 1;  
    if (n >= 5) return n;  
    n = n + i;  
    i++;  
    return f(n);  
}
```

The value returned by $f(1)$ is:

A. 5

B. 6

C. 7

D. 8

gatecse-2004 programming programming-in-c recursion easy isro2008

Answer key

5.13.7 Recursion: GATE CSE 2005 | Question: 81b



```
double foo(int n)  
{  
    int i;  
    double sum;  
    if (n == 0)  
    {  
        return 1.0;  
    }  
    else  
    {  
        sum = 0.0;  
        for (i = 0; i < n; i++)  
        {  
            sum += foo(i);  
        }  
        return sum;  
    }  
}
```

Suppose we modify the above function $foo()$ and stores the value of $foo(i)$ $0 \leq i < n$ as and when they are computed. With this modification the time complexity for function $foo()$ is significantly reduced. The space complexity of the modified function would be:

A. $O(1)$

B. $O(n)$

C. $O(n^2)$

D. $n!$

gatecse-2005 programming recursion normal

Answer key

5.13.8 Recursion: GATE CSE 2007 | Question: 42



Consider the following C function:

```
int f(int n)
{
    static int r = 0;
    if (n <= 0) return 1;
    if (n > 3)
    {
        r = n;
        return f(n-2) + 2;
    }
    return f(n-1) + r;
}
```

What is the value of $f(5)$?

A. 5

B. 7

C. 9

D. 18

gatecse-2007 programming recursion normal

Answer key

5.13.9 Recursion: GATE CSE 2014 | Set 2 | Question: 40



Consider the following function.

```
double f(double x){
    if (abs(x*x - 3) < 0.01)
        return x;
    else
        return f(x/2 + 1.5/x);
}
```

Give a value q (to 2 decimals) such that $f(q)$ will return q : _____.

gatecse-2014-set2 programming recursion numerical-answers normal

Answer key

5.13.10 Recursion: GATE CSE 2015 | Set 2 | Question: 15



Consider the following function written in the C programming language:

```
void foo(char *a)
{
    if (*a && *a != '\0')
    {
        foo(a+1);
        putchar(*a);
    }
}
```

The output of the above function on input "ABCD EFGH" is

A. ABCD EFGH

B. ABCD

C. HGFE DCBA

D. DCBA

gatecse-2015-set2 programming programming-in-c normal recursion

Answer key

5.13.11 Recursion: GATE CSE 2016 | Set 1 | Question: 35



What will be the output of the following C program?

```
void count (int n) {
    static int d=1;

    printf ("%d",n);
    printf ("%d",d);
    d++;
    if (n>1) count (n-1);
    printf ("%d",d);
}
```

```
void main(){
    count(3);
}
```

- A. 3 1 2 2 1 3 4 4 4
- B. 3 1 2 1 1 1 2 2 2
- C. 3 1 2 2 1 3 4
- D. 3 1 2 1 1 1 2

gatecse-2016-set1 programming-in-c recursion normal

Answer key

5.13.12 Recursion: GATE CSE 2016 | Set 2 | Question: 37



Consider the following program:

```
int f (int *p, int n)
{
    if (n <= 1) return 0;
    else return max (f (p+1, n-1), p[0] - p[1]);
}

int main ()
{
    int a[] = {3, 5, 2, 6, 4};
    printf ("%d", f(a, 5));
}
```

Note: $\max(x, y)$ returns the maximum of x and y .

The value printed by this program is _____.

gatecse-2016-set2 programming-in-c normal numerical-answers recursion

Answer key

5.13.13 Recursion: GATE CSE 2017 | Set 1 | Question: 35



Consider the following two functions.

```
void fun1(int n) {
    if (n == 0) return;
    printf ("%d" n);
    fun2(n-2);
    printf ("%d" n);
}

void fun2(int n) {
    if (n == 0) return;
    printf ("%d" n);
    fun1(++n);
    printf ("%d" n);
}
```

The output printed when fun1(5) is called is

- A. 53423122233445
- B. 53423120112233
- C. 53423122132435
- D. 53423120213243

gatecse-2017-set1 programming normal tricky recursion

Answer key

5.13.14 Recursion: GATE CSE 2017 | Set 1 | Question: 36



Consider the C functions foo and bar given below:

```
int foo(int val) {
    int x=0;
    while(val > 0) {
        x = x + foo(val--);
    }
}
```

```
}  
return val;  
}
```

```
int bar(int val) {  
    int x = 0;  
    while(val > 0) {  
        x = x + bar(val-1);  
    }  
    return val;  
}
```

Invocations of `foo(3)` and `bar(3)` will result in:

- A. Return of 6 and 6 respectively
- B. Infinite loop and abnormal termination respectively.
- C. Abnormal termination and infinite loop respectively.
- D. Both terminating abnormally.

gatecse-2017-set1 programming-in-c programming normal recursion

Answer key

5.13.15 Recursion: GATE CSE 2018 | Question: 21



Consider the following C program:

```
#include<stdio.h>  
  
int counter=0;  
  
int calc (int a, int b) {  
    int c;  
    counter++;  
    if(b==3) return (a*a*a);  
    else {  
        c = calc(a, b/3);  
        return (c*c*c);  
    }  
}  
  
int main() {  
    calc(4, 81);  
    printf("%d", counter);  
}
```

The output of this program is _____.

gatecse-2018 programming-in-c numerical-answers recursion programming one-mark

Answer key

5.13.16 Recursion: GATE CSE 2020 | Question: 46



Consider the following C functions.

```
int fun1(int n) {  
    static int i= 0;  
    if (n > 0) {  
        ++i;  
        fun1(n-1);  
    }  
    return (i);  
}
```

```
int fun2(int n) {  
    static int i= 0;  
    if (n>0) {  
        i = i+ fun1 (n) ;  
        fun2(n-1) ;  
    }  
    return (i);  
}
```

The return value of `fun2(5)` is _____

gatecse-2020 numerical-answers programming-in-c recursion two-marks

Answer key

5.13.17 Recursion: GATE CSE 2025 | Set 1 | Question: 51



```
#include <stdio.h>
```

```
int foo(int S[],int size) {
    if(size == 0) return 0;
    if(size == 1) return 1;
    if(S[0] != S[1]) return 1+foo(S+1,size-1);
    return foo(S+1,size-1);
}
int main() {
    int A[]={0,1,2,2,2,0,0,1,1};
    printf("%d",foo(A,9));
    return 0;
}
```

The value printed by the given C program is _____. (Answer in integer)

gatecse2025-set1 programming-in-c recursion output numerical-answers two-marks

Answer key

5.13.18 Recursion: GATE CSE 2026 | Set 2 | Question: 51



Consider the following ANSI-C function.

```
int func(int start, int end){
    int length=end+1-start;
    if((length<1)|| (start<0)|| (end<0)){ return(0); }
    if(length%3==0){
        return(func(start+1, end));
    }else if(length%3==1){
        return(1+func(start, end-1));
    }else {
        return(func(start+2, end));
    }
}
```

The maximum possible value that can be returned from this function is _____. (answer in integer)

Note: Ignore syntax errors (if any) in the function.

gatecse-2026-set2 programming-in-c recursion output numerical-answers two-marks

Answer key

5.13.19 Recursion: GATE IT 2007 | Question: 27



The function f is defined as follows:

```
int f (int n) {
    if (n <= 1) return 1;
    else if (n % 2 == 0) return f(n/2);
    else return f(3n - 1);
}
```

Assuming that arbitrarily large integers can be passed as a parameter to the function, consider the following statements.

- The function f terminates for finitely many different values of $n > 1$.
- The function f terminates for infinitely many different values of $n > 1$.
- The function f does not terminate for finitely many different values of $n > 1$.
- The function f does not terminate for infinitely many different values of $n > 1$.

Which one of the following options is true of the above?

- A. i and iii B. i and iv C. ii and iii D. ii and iv

gateit-2007 programming recursion normal

Answer key

5.14

Runtime Environment (1)

5.14.1 Runtime Environment: GATE CSE 2026 | Set 2 | Question: 7



In C runtime environment, which one of the following is stored in heap?

- A. A static variable declared inside a function
- B. An array of integers declared inside a function
- C. A dynamically allocated array of integers created using malloc() function call
- D. Return address of a function

gatecse-2026-set2 programming-in-c runtime-environment one-mark

Answer key

5.15 Strings (2)

5.15.1 Strings: GATE CSE 2004 | Question: 33



Consider the following C program segment:

```
char p[20]; int i;
char* s = "string";
int length = strlen(s);
for(i = 0; i < length; i++)
    p[i] = s[length-i];
printf("%s", p);
```

The output of the program is:

- A. gnirts
- B. string
- C. gnirt
- D. no output is printed

gatecse-2004 programming programming-in-c strings easy

Answer key

5.15.2 Strings: GATE CSE 2025 | Set 2 | Question: 9



Consider the following C program:

```
#include <stdio.h>
void stringcopy(char *, char *);
int main() {
    char a[30] = "@#Hello World!";
    stringcopy(a, a+2);
    printf("%s\n", a);
    return 0;
}
void stringcopy(char *s, char *t) {
    while (*t)
        *s++ = *t++;
}
```

Which one of the following will be the output of the program?

- A. @#Hello World!
- B. Hello World!
- C. ello World!
- D. Hello World!d!

gatecse2025-set2 programming-in-c strings output one-mark

Answer key

5.16 Structure (5)

5.16.1 Structure: GATE CSE 2000 | Question: 1.11



The following C declarations:

```
struct node {
    int i;
    float j;
};
struct node *s[10];
```

define s to be:

- A. An array, each element of which is a pointer to a structure of type node
- B. A structure of 2 fields, each field being a pointer to an array of 10 elements
- C. A structure of 3 fields: an integer, a float, and an array of 10 elements
- D. An array, each element of which is a structure of type node

gatecse-2000 programming programming-in-c easy structure

Answer key

5.16.2 Structure: GATE CSE 2018 | Question: 2



Consider the following C program:

```
#include<stdio.h>
struct Ournode{
    char x, y, z;
};
int main() {
    struct Ournode p=('1', '0', 'a'+2);
    struct Ournode *q=&p;
    printf("%c, %c" *((char*)q+1), *((char*)q+2));
    return 0;
}
```

The output of this program is:

- A. 0, c B. 0, a+2 C. '0', 'a+2' D. '0', 'c'

gatecse-2018 programming-in-c programming structure normal one-mark

Answer key

5.16.3 Structure: GATE CSE 2021 | Set 2 | Question: 35



Consider the following ANSI C program:

```
#include <stdio.h>
#include <stdlib.h>
struct Node{
    int value;
    struct Node *next;
};
int main() {
    struct Node *boxE, *head, *boxN; int index=0;
    boxE=head= (struct Node *) malloc(sizeof(struct Node));
    head->value=index;
    for (index=1; index<=3; index++) {
        boxN= (struct Node *) malloc (sizeof(struct Node));
        boxE->next=boxN;
        boxN->value=index;
        boxE=boxN;
    }
    for (index=0; index<=3; index++) {
        printf("Value at index %d is %d\n", index, head->value);
        head=head->next;
        printf("Value at index %d is %d\n", index+1, head->value); }
}
```

Which one of the following statements below is correct about the program?

- A. Upon execution, the program creates a linked-list of five nodes
B. Upon execution, the program goes into an infinite loop
C. It has a missing return which will be reported as an error by the compiler
D. It dereferences an uninitialized pointer that may result in a run-time error

gatecse-2021-set2 programming-in-c normal pointers structure two-marks

Answer key

5.16.4 Structure: GATE IT 2004 | Question: 61



Consider the following C program:

```
#include <stdio.h>
typedef struct {
    char *a;
    char *b;
} t;
void f1 (t s);
void f2 (t *p);
main()
```



```

static t s = {"A", "B"};
printf ("%s %s\n", s.a, s.b);
f1(s);
printf ("%s %s\n", s.a, s.b);
f2(&s);

void f1 (t s)
{
    s.a = "U";
    s.b = "V";
    printf ("%s %s\n", s.a, s.b);
    return;
}

void f2(t *p)
{
    p->a = "V";
    p->b = "W";
    printf ("%s %s\n", p->a, p->b);
    return;
}

```

What is the output generated by the program ?

A. A B
U V
V W
V W

C. A B
U V
U V
V W

B. A B
U V
A B
V W

D. A B
U V
V W
U V

gateit-2004 programming programming-in-c normal structure

Answer key

5.16.5 Structure: GATE IT 2006 | Question: 49

Which one of the choices given below would be printed when the following program is executed ?

```

#include <stdio.h>
struct test {
    int i;
    char *c;
} st[] = {5, "become", 4, "better", 6, "jungle", 8, "ancestor", 7, "brother"};
main ()
{
    struct test *p = st;
    p += 1;
    ++p->c;
    printf ("%s", p++->c);
    printf ("%c", *++p->c);
    printf ("%d", p[0].i);
    printf ("%s \n", p->c);
}

```

A. jungle, n, 8, nclastor
C. cetter, k, 6, jungle

B. etter, u, 6, ungle
D. etter, u, 8, ncestor

gateit-2006 programming programming-in-c normal structure

Answer key

5.17

Switch Case (2)

5.17.1 Switch Case: GATE CSE 2012 | Question: 3

What will be the output of the following C program segment?

```

char inChar = 'A';
switch ( inChar ) {
    case 'A' : printf ("Choice A \n");
    case 'B' :
    case 'C' : printf ("Choice B");
    case 'D' :
    case 'E' :
    default : printf ("No Choice");
}

```

```
}  
}
```

A. No Choice

C. Choice A

Choice B No Choice

B. Choice A

D. Program gives no output as it is erroneous

gatecse-2012 programming easy programming-in-c switch-case marks-to-all

Answer key

5.17.2 Switch Case: GATE CSE 2015 | Set 3 | Question: 48



Consider the following C program:

```
#include<stdio.h>  
int main()  
{  
    int i, j, k = 0;  
    j = 2 * 3 / 4 + 2.0 / 5 + 8 / 5;  
    k = --j;  
    for (i = 0; i < 5; i++)  
    {  
        switch (i + k)  
        {  
            case 1:  
            case 2: printf("\n%d", i + k);  
            case 3: printf("\n%d", i + k);  
            default: printf("\n%d", i + k);  
        }  
    }  
    return 0;  
}
```

The number of times printf statement is executed is _____.

gatecse-2015-set3 programming programming-in-c switch-case normal numerical-answers

Answer key

5.18

Type Checking (1)

5.18.1 Type Checking: GATE CSE 2003 | Question: 24



Which of the following statements is FALSE?

- A. In statically typed languages, each variable in a program has a fixed type
- B. In un-typed languages, values do not have any types
- C. In dynamically typed languages, variables have no types
- D. In all statically typed languages, each variable in a program is associated with values of only a single type during the execution of the program

gatecse-2003 programming normal type-checking

Answer key

5.19

Union (1)

5.19.1 Union: GATE CSE 2000 | Question: 1.17, ISRO2015-79



Consider the following C declaration:

```
struct {  
    short x[5];  
    union {  
        float y;  
        long z;  
    } u;  
} t;
```

Assume that the objects of the type short, float and long occupy 2 bytes, 4 bytes and 8 bytes, respectively. The memory requirement for variable t , ignoring alignment consideration, is:

A. 22 bytes

B. 14 bytes

C. 18 bytes

D. 10 bytes

Answer key

5.20

Variable Binding (1)

5.20.1 Variable Binding: GATE IT 2007 | Question: 34, UGCNET-Dec2012-III: 52



Consider the program below in a hypothetical programming language which allows global variables and a choice of static or dynamic scoping.

```
int i;
program main()
{
    i = 10;
    call f();
}

procedure f()
{
    int i = 20;
    call g();
}

procedure g()
{
    print i;
}
```

Let x be the value printed under static scoping and y be the value printed under dynamic scoping. Then, x and y are:

A. $x = 10, y = 20$ B. $x = 20, y = 10$ C. $x = 10, y = 10$ D. $x = 20, y = 20$

Answer key

Answer Keys

5.1.1	A	5.2.1	B	5.2.2	C	5.2.3	A	5.2.4	C
5.2.5	C	5.2.6	6	5.2.7	19	5.2.8	C	5.2.9	A
5.2.10	C	5.2.11	B	5.2.12	C	5.2.13	B	5.3.1	230
5.3.2	A	5.4.1	A	5.4.2	A	5.5.1	N/A	5.5.2	C
5.5.3	C	5.5.4	3	5.5.5	26	5.5.6	B	5.6.1	N/A
5.6.2	N/A	5.6.3	N/A	5.6.4	N/A	5.6.5	A	5.6.6	B
5.6.7	C	5.6.8	C	5.7.1	A	5.7.2	7	5.7.3	C
5.7.4	C	5.7.5	46:46	5.7.6	21:21	5.7.7	46:46	5.7.8	09 :09
5.8.1	N/A	5.8.2	C	5.8.3	B	5.8.4	D	5.8.5	B
5.8.6	D	5.8.7	6561	5.8.8	2016	5.8.9	30	5.8.10	A
5.8.11	D	5.8.12	C	5.9.1	D	5.9.2	C	5.9.3	A
5.9.4	A	5.9.5	C	5.9.6	C	5.9.7	D	5.9.8	140
5.9.9	D	5.9.10	2	5.9.11	D	5.9.12	B	5.9.13	25:25
5.9.14	111:111	5.9.15	9:9	5.10.1	B	5.11.1	A	5.11.2	B
5.11.3	B	5.11.4	D	5.11.5	A	5.11.6	D	5.11.7	C
5.11.8	D	5.11.9	B	5.11.10	D	5.11.11	-5	5.11.12	D
5.11.13	D	5.11.14	3	5.11.15	23	5.11.16	A	5.11.17	0
5.11.18	B	5.11.19	B	5.11.20	5	5.11.21	10	5.11.22	B

5.11.23	B;D	5.11.24	A	5.11.25	A	5.11.26	D	5.11.27	B
5.11.28	A	5.11.29	C	5.12.1	C	5.12.2	D	5.13.1	41
5.13.2	16	5.13.3	N/A	5.13.4	N/A	5.13.5	N/A	5.13.6	C
5.13.7	B	5.13.8	D	5.13.9	1.72 : 1.74	5.13.10	D	5.13.11	A
5.13.12	3	5.13.13	A	5.13.14	C	5.13.15	4	5.13.16	55
5.13.17	5:5	5.13.18	1:1	5.13.19	D	5.14.1	C	5.15.1	D
5.15.2	D	5.16.1	A	5.16.2	A	5.16.3	D	5.16.4	B
5.16.5	B	5.17.1	X	5.17.2	10	5.18.1	C	5.19.1	C
5.20.1	A								



Regular expressions and finite automata, Context-free grammars and push-down automata, Regular and context-free languages, Pumping lemma, Turing machines and undecidability.

Mark Distribution in Previous GATE

Year	2026 - 1	2026 - 2	2025 - 1	2025 - 2	2024 - 1	2024 - 2	2023	2022	2021 - 1	2021 - 2	Minimum
1 Mark Count	2	1	2	3	1	1	3	2	2	3	1
2 Marks Count	2	2	4	2	2	3	3	3	3	4	2
Total Marks	6	5	10	7	5	7	9	8	8	11	5

Welcome to the "Theory of Computation" chapter, a cornerstone of Computer Science that delves into the fundamental capabilities and limitations of computational models. This subject is crucial for the GATE CS exam as it builds a strong theoretical foundation for understanding algorithms, programming languages, and system design. It typically carries a weightage of 8-12 marks, with questions ranging from conceptual understanding of language classes and machine models to problem-solving involving the design of automata, grammar conversions, and decidability proofs. Expect a mix of Multiple Choice Questions (MCQs), Multiple Select Questions (MSQs), and Numerical Answer Type (NAT) questions, often requiring a deep grasp of definitions, properties, and theorems.

Topic-wise Key Concepts

Finite Automata (FA)

Finite Automata are the simplest computational models, used to recognize regular languages. They have a finite number of states and transitions based on input symbols, without any auxiliary memory. They are fundamental for lexical analysis in compilers and designing simple control systems.

- **Definition:** A Deterministic Finite Automaton (DFA) is formally defined as a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where:
 1. Q is a finite set of states.
 2. Σ is a finite set of input symbols (alphabet).
 3. $\delta : Q \times \Sigma \rightarrow Q$ is the transition function.
 4. $q_0 \in Q$ is the initial state.
 5. $F \subseteq Q$ is the set of final (accepting) states.
- **Key Properties:**
 - Every NFA has an equivalent DFA.
 - DFA and NFA recognize the same class of languages (Regular Languages).
 - DFA can be minimized to a unique (up to isomorphism) minimal state DFA.
- **Common Pitfalls:** Incorrectly handling epsilon transitions in NFA to DFA conversion; confusing acceptance criteria for NFA/DFA.
- **Problem-Solving Techniques:**
 - **Subset Construction:** For converting NFA to DFA.
 - **Table Filling Algorithm / Myhill-Nerode Theorem:** For DFA minimization.
 - **State Elimination Method / Arden's Theorem:** For converting FA to Regular Expression.

Finite State Machines (FSM)

Finite State Machines are mathematical models of computation used to design systems that can be in one of a finite number of states. While often used interchangeably with FA, FSMs typically include an output function, making them suitable for modeling sequential circuits and control logic.

- **Definition:**
 - **Mealy Machine:** $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where Δ is the output alphabet, and $\lambda : Q \times \Sigma \rightarrow \Delta$ is the output function. Output depends on current state and input.
 - **Moore Machine:** $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where $\lambda : Q \rightarrow \Delta$ is the output function. Output depends only on the current state.
- **Key Properties:**
 - Any Mealy machine can be converted to an equivalent Moore machine, and vice-versa.
 - Moore machine output is delayed by one clock cycle compared to Mealy.
- **Common Pitfalls:** Incorrectly converting between Mealy and Moore machines, especially handling initial state outputs.
- **Problem-Solving Techniques:** State diagram construction, state table analysis, conversion algorithms between

Mealy and Moore.

Regular Expression (RE)

Regular Expressions are a powerful notation for describing regular languages. They provide a concise way to specify patterns of strings, widely used in text processing, search engines, and lexical analysis.

- **Definition:** A Regular Expression is built using basic symbols and three operations:
 - **Union:** $R_1 + R_2$ (or $R_1 | R_2$) represents strings in $L(R_1)$ or $L(R_2)$.
 - **Concatenation:** $R_1 R_2$ represents strings formed by concatenating a string from $L(R_1)$ with one from $L(R_2)$.
 - **Kleene Star:** R^* represents zero or more concatenations of strings from $L(R)$, including the empty string ϵ .
- **Key Identities:**
 - $R + \emptyset = R$
 - $R\epsilon = \epsilon R = R$
 - $(R^*)^* = R^*$
 - $\emptyset^* = \epsilon$
 - $R(S + T) = RS + RT$
 - $(RS)^* R = R(SR)^*$
 - $(R + S)^* = (R^* S^*)^* = (R^* + S^*)^*$
- **Common Pitfalls:** Operator precedence (Kleene star > concatenation > union); incorrect application of identities.
- **Problem-Solving Techniques:**
 - **Arden's Theorem:** For solving equations of the form $R = Q + RP$ to get $R = QP^*$.
 - **State Elimination Method:** For converting FA to RE.

Regular Grammar (RG)

Regular Grammars are a type of formal grammar that generates exactly the class of regular languages. They are restricted in their production rules, making them less powerful than Context-Free Grammars but simpler to parse.

- **Definition:** A grammar $G = (V, \Sigma, P, S)$ is regular if all its production rules are of one of the following forms:
 - **Right-linear:** $A \rightarrow aB$ or $A \rightarrow a$ or $A \rightarrow \epsilon$, where $A, B \in V$ (non-terminals), $a \in \Sigma$ (terminal).
 - **Left-linear:** $A \rightarrow Ba$ or $A \rightarrow a$ or $A \rightarrow \epsilon$, where $A, B \in V$, $a \in \Sigma$.
- **Key Properties:**
 - A grammar cannot be both right-linear and left-linear simultaneously (unless it generates a finite language).
 - Regular Grammars, Regular Expressions, and Finite Automata are all equivalent in terms of language recognition power.
- **Common Pitfalls:** Mixing right-linear and left-linear rules in the same grammar; incorrectly converting between FA and RG.
- **Problem-Solving Techniques:** Constructing an FA from a RG, and vice-versa, by mapping non-terminals to states and productions to transitions.

Regular Language (RL)

Regular Languages are the simplest class of languages in the Chomsky Hierarchy, recognized by Finite Automata, described by Regular Expressions, or generated by Regular Grammars. They represent patterns that can be matched without requiring memory beyond the current state.

- **Definition:** A language is regular if and only if it can be recognized by a Finite Automaton.
- **Key Properties (Closure Properties):** Regular languages are closed under:
 - Union ($L_1 \cup L_2$)
 - Intersection ($L_1 \cap L_2$)
 - Complement ($\Sigma^* \setminus L$)
 - Concatenation ($L_1 L_2$)
 - Kleene Star (L^*)
 - Reversal (L^R)
 - Homomorphism, Inverse Homomorphism
- **Common Pitfalls:** Misidentifying non-regular languages as regular; incorrectly applying closure properties.
- **Problem-Solving Techniques:**
 - **Pumping Lemma for Regular Languages:** The primary tool to prove a language is NOT regular.
 - **Myhill-Nerode Theorem:** Can be used to prove regularity or non-regularity, and for DFA minimization.

Non Determinism

Non-determinism in automata allows for multiple possible transitions from a given state on a given input symbol, or transitions on an empty string (epsilon transitions). It often simplifies the design of automata, though it doesn't increase their computational power for regular languages.

- **Definition:** A Non-deterministic Finite Automaton (NFA) is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$ is the transition function, mapping to a set of states.
- **Key Properties:**
 - ◻ Every NFA has an equivalent DFA (recognizes the same language).
 - ◻ An NFA can be exponentially more concise than its equivalent DFA (e.g., an NFA with n states might require up to 2^n states in the equivalent DFA).
 - ◻ Non-determinism is crucial for Pushdown Automata (NPDA is more powerful than DPDA).
- **Common Pitfalls:** Incorrectly handling epsilon closures; missing possible paths in non-deterministic transitions.
- **Problem-Solving Techniques:**
 - ◻ **Subset Construction Algorithm:** The standard method to convert an NFA (with or without epsilon transitions) to an equivalent DFA.
 - ◻ Understanding how to trace paths in an NFA to determine language acceptance.

Number of States

The "number of states" refers to the cardinality of the set of states Q in an automaton. This concept is critical for understanding the complexity and minimality of machine models, particularly for finite automata.

- **Key Properties:**
 - ◻ For an NFA with n states, the equivalent DFA can have up to 2^n states.
 - ◻ A minimal DFA for a given regular language is unique (up to isomorphism) and has the fewest possible states.
 - ◻ The number of states in a minimal DFA is equal to the number of equivalence classes of the Myhill-Nerode relation.
- **Common Pitfalls:** Failing to identify redundant or unreachable states, which leads to non-minimal DFAs.
- **Problem-Solving Techniques:**
 - ◻ **DFA Minimization Algorithms:** Partitioning states into distinguishable/indistinguishable sets.
 - ◻ **Myhill-Nerode Theorem:** Using equivalence classes of strings to determine the minimum number of states.

Minimal State Automata

A Minimal State Automaton (specifically, a Minimal DFA) is a Deterministic Finite Automaton that recognizes a given regular language using the smallest possible number of states. It is unique for any given regular language and is crucial for efficient implementation.

- **Definition:** A DFA M' is minimal if there is no other DFA M'' such that $L(M'') = L(M')$ and $|Q''| < |Q'|$.
- **Key Properties:**
 - ◻ Every regular language has a unique minimal DFA (up to isomorphism).
 - ◻ All states in a minimal DFA must be reachable from the start state and must be distinguishable from each other.
- **Common Pitfalls:** Incorrectly merging distinguishable states; not removing unreachable states before minimization.
- **Problem-Solving Techniques:**
 - ◻ **Table Filling Algorithm (or Partitioning Algorithm):**
 1. Initialize a table of all pairs of states (p, q) . Mark all pairs (p, q) where $p \in F$ and $q \notin F$ (or vice versa) as distinguishable.
 2. Iteratively mark (p, q) as distinguishable if for some input symbol a , $(\delta(p, a), \delta(q, a))$ is already marked distinguishable.
 3. Repeat until no new pairs can be marked. All unmarked pairs are indistinguishable and can be merged.
 - ◻ **Myhill-Nerode Theorem:** States that a language L is regular if and only if the number of equivalence classes of the Myhill-Nerode relation $\equiv_L$ is finite. The number of states in the minimal DFA is equal to the number of these equivalence classes.

Pumping Lemma (for Regular Languages)

The Pumping Lemma for Regular Languages is a fundamental theorem used to prove that a language is NOT regular. It states that all sufficiently long strings in a regular language can be "pumped" (i.e., a middle section of the string can be repeated any number of times) and the resulting string will still be in the language.

- **Theorem:** If L is a regular language, then there exists some integer $p > 1$ (the pumping length) such that for any string $s \in L$ with $|s| \geq p$, s can be divided into three parts $s = xyz$ satisfying the conditions:
 1. $|y| > 0$ (the middle part cannot be empty).
 2. $|xy| \leq p$ (the pumpable part must occur within the first p symbols).
 3. For all $i \geq 0$, the string $xy^iz \in L$.
- **Key Properties:**
 - It is a necessary condition for regularity, not a sufficient one. It can only be used to prove non-regularity.
 - The proof strategy is usually by contradiction (an "adversary game").
- **Common Pitfalls:**
 - Trying to prove regularity using the Pumping Lemma (it's impossible).
 - Incorrectly choosing the string s or the partition xyz to break the lemma.
 - Forgetting the condition $|y| > 0$ or $|xy| \leq p$.
- **Problem-Solving Techniques:**
 1. Assume L is regular and let p be the pumping length.
 2. Choose a "hard" string $s \in L$ such that $|s| \geq p$, typically one that grows with p (e.g., $a^p b^p$).
 3. Consider all possible ways to divide s into xyz satisfying $|y| > 0$ and $|xy| \leq p$.
 4. For each division, find an $i \geq 0$ such that $xy^iz \notin L$. This contradicts the lemma, proving L is not regular.

Closure Property

Closure properties describe whether a class of languages remains within that class after applying certain operations. Understanding these properties is vital for classifying languages and proving relationships between language classes.

- **Definition:** A class of languages $\mathcal{C}$ is closed under an operation $\mathcal{O}$ if, for any language $L_1, L_2 \in \mathcal{C}$, the result of $\mathcal{O}(L_1, L_2)$ (or $\mathcal{O}(L_1)$) is also in $\mathcal{C}$.
- **Key Properties (to memorize):**

Operation	Regular Languages	Context-Free Languages	Recursive Languages	Recursively Enumerable Languages
Union	Yes	Yes	Yes	Yes
Intersection	Yes	No	Yes	Yes
Complement	Yes	No	Yes	No
Concatenation	Yes	Yes	Yes	Yes
Kleene Star	Yes	Yes	Yes	Yes
Reversal	Yes	Yes	Yes	Yes
Homomorphism	Yes	Yes	Yes	Yes
Inverse Homomorphism	Yes	Yes	Yes	Yes

- **Common Pitfalls:** Confusing closure properties between different language classes, especially for intersection and complement.
- **Problem-Solving Techniques:** Use closure properties to deduce the class of a new language based on known languages and operations. For non-closure, use counterexamples.

Context Free Grammar (CFG)

Context-Free Grammars are more powerful than regular grammars and are used to describe Context-Free Languages. They are fundamental to defining the syntax of programming languages and parsing. Productions allow a non-terminal to be replaced by a string of terminals and non-terminals, regardless of its context.

- **Definition:** A CFG is a 4-tuple $G = (V, \Sigma, P, S)$, where:
 1. V is a finite set of non-terminal symbols.
 2. Σ is a finite set of terminal symbols.
 3. P is a finite set of production rules of the form $A \rightarrow \alpha$ where $A \in V$ and $\alpha \in (V \cup \Sigma)^*$.
 4. $S \in V$ is the start symbol.
- **Key Forms:**
 - **Chomsky Normal Form (CNF):** All productions are of the form $A \rightarrow BC$ or $A \rightarrow a$ where $A, B, C \in V$ and $a \in \Sigma$. Productions are allowed only for the start symbol if it doesn't appear on the RHS.
 - **Greibach Normal Form (GNF):** All productions are of the form $A \rightarrow a\alpha$, where $A \in V$, $a \in \Sigma$, and $\alpha \in V^*$.
- **Common Pitfalls:** Detecting ambiguity (a string having more than one leftmost/rightmost derivation or parse tree); incorrect conversion to CNF/GNF.

- **Problem-Solving Techniques:**

- **Simplification of CFG:** Removing useless symbols, ϵ -productions, and unit productions.
- **Conversion to CNF/GNF:** Systematic procedures involving variable substitution and rule modification.
- Derivation trees, leftmost/rightmost derivations.

Context Free Language (CFL)

Context-Free Languages are recognized by Pushdown Automata and generated by Context-Free Grammars. They can describe languages with nested structures, such as balanced parentheses or properly nested HTML tags, which regular languages cannot handle.

- **Definition:** A language is Context-Free if and only if it can be generated by a Context-Free Grammar.

- **Key Properties:**

- **Closure Properties:** Closed under union, concatenation, Kleene star, reversal, homomorphism, inverse homomorphism.
- **Non-Closure Properties:** NOT closed under intersection, complement.
- Deterministic CFLs (DCFLs) are a proper subset of CFLs and are closed under complement.

- **Common Pitfalls:** Misidentifying non-CFLs as CFLs; incorrectly applying closure properties.

- **Problem-Solving Techniques:**

- **Pumping Lemma for CFLs:** The primary tool to prove a language is NOT context-free.
- Designing CFGs or PDAs for given languages.

Pushdown Automata (PDA)

Pushdown Automata are an extension of Finite Automata with an additional stack memory. This stack allows PDAs to recognize Context-Free Languages, which require memory to handle nested structures, unlike regular languages.

- **Definition:** A Pushdown Automaton is formally defined as a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where:

1. Q is a finite set of states.
2. Σ is the input alphabet.
3. Γ is the stack alphabet.
4. $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$ is the transition function.
5. $q_0 \in Q$ is the initial state.
6. $Z_0 \in \Gamma$ is the initial stack symbol.
7. $F \subseteq Q$ is the set of final states.

- **Key Properties:**

- Non-deterministic PDAs (NPDAs) are more powerful than Deterministic PDAs (DPDAs). NPDAs recognize all CFLs, while DPDAs recognize only DCFLs (a proper subset of CFLs).
- PDAs can accept languages either by final state or by empty stack. These two modes are equivalent for NPDAs.

- **Common Pitfalls:** Incorrectly handling stack operations (push/pop); confusing acceptance criteria; designing a DPDA when an NPDA is required.

- **Problem-Solving Techniques:**

- Designing PDAs for given CFLs, focusing on how the stack is used to remember context.
- Converting CFG to PDA and vice-versa.

Dpda (Deterministic Pushdown Automata)

A Deterministic Pushdown Automaton (DPDA) is a PDA where for any given state, input symbol (or epsilon), and stack top, there is at most one possible transition. DPDAs recognize Deterministic Context-Free Languages (DCFLs), which are a proper subset of CFLs.

- **Definition:** A PDA is deterministic if its transition function δ satisfies:

1. For any $(q, a, X) \in Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma$, $|\delta(q, a, X)| \leq 1$.
2. If $\delta(q, \epsilon, X) \neq \emptyset$, then $\delta(q, a, X) = \emptyset$ for all $a \in \Sigma$ (No conflict between epsilon and input transitions).

- **Key Properties:**

- DPDAs recognize DCFLs. All regular languages are DCFLs.
- DCFLs are closed under complement (unlike general CFLs).
- Not all CFLs are DCFLs (e.g., $\{ww^R \mid w \in \{a, b\}^*\}$ is a CFL but not a DCFL).

- **Common Pitfalls:** Failing to ensure determinism in all transition cases; confusing DCFLs with general CFLs.

- **Problem-Solving Techniques:** Designing DPDAs requires careful consideration to avoid non-deterministic choices. Often involves lookahead or specific stack symbol usage.

Pumping Lemma (for CFLs)

The Pumping Lemma for Context-Free Languages is a more generalized version of the Pumping Lemma for Regular Languages. It's used to prove that certain languages are NOT context-free, by showing that sufficiently long strings in a CFL can be "pumped" in a specific way.

- **Theorem:** If L is a Context-Free Language, then there exists some integer $p \geq 1$ (the pumping length) such that for any string $s \in L$ with $|s| \geq p$, s can be divided into five parts $s = uvwxy$ satisfying the conditions:
 1. $|vx| > 0$ (at least one of v or x must be non-empty).
 2. $|vwx| \leq p$ (the pumpable parts must occur relatively close to each other).
 3. For all $i \geq 0$, the string $uv^iwx^iy \in L$.
- **Key Properties:**
 - It is a necessary condition for context-freeness, not a sufficient one. It can only be used to prove non-context-freeness.
 - The proof strategy is by contradiction (an "adversary game"), similar to the regular pumping lemma.
- **Common Pitfalls:**
 - Trying to prove context-freeness using the Pumping Lemma.
 - Incorrectly choosing the string s or the partition $uvwxy$.
 - Forgetting the conditions $|vx| > 0$ or $|vwx| \leq p$.
- **Problem-Solving Techniques:**
 1. Assume L is CFL and let p be the pumping length.
 2. Choose a "hard" string $s \in L$ such that $|s| \geq p$, typically one that involves three dependent counts (e.g., $a^p b^p c^p$).
 3. Consider all possible ways to divide s into $uvwxy$ satisfying $|vx| > 0$ and $|vwx| \leq p$.
 4. For each division, find an $i \geq 0$ such that $uv^iwx^iy \notin L$. This contradicts the lemma, proving L is not CFL.

Turing Machine (TM)

The Turing Machine is the most powerful and widely accepted model of computation, capable of simulating any algorithm. It consists of a finite control unit, a read/write head, and an infinite tape. It forms the basis of the Church-Turing Thesis, which states that any effectively computable function can be computed by a Turing Machine.

- **Definition:** A Turing Machine is formally defined as a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where:
 1. Q is a finite set of states.
 2. Σ is the input alphabet (not containing B).
 3. Γ is the tape alphabet ($\Sigma \subset \Gamma$, and $B \in \Gamma$).
 4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function (L for left, R for right).
 5. $q_0 \in Q$ is the initial state.
 6. $B \in \Gamma$ is the blank symbol.
 7. $F \subseteq Q$ is the set of final (halting) states.
- **Key Properties:**
 - Turing Machines can recognize Recursively Enumerable Languages.
 - Variations of TMs (multi-tape, multi-head, non-deterministic) have equivalent computational power to the basic TM.
 - The Halting Problem for Turing Machines is undecidable.
- **Common Pitfalls:** Designing complex TMs; understanding the implications of the halting problem; confusing decidability with recognizability.
- **Problem-Solving Techniques:**
 - Designing TMs for various languages (e.g., $\{a^n b^n c^n\}$, palindrome recognition).
 - Understanding the Church-Turing Thesis and its implications for computability.

Recursive and Recursively Enumerable Languages

These two classes of languages represent the limits of what Turing Machines can compute. They are central to the study of computability and decidability.

- **Definition:**
 - **Recursively Enumerable Language (RE / Recognizable):** A language L is RE if there exists a Turing Machine M that halts and accepts every string $w \in L$, but may either halt and reject or loop indefinitely for strings $w \notin L$.

- **Recursive Language (Decidable):** A language L is Recursive if there exists a Turing Machine M that halts on all inputs. For $w \in L$, M accepts; for $w \notin L$, M rejects.
- **Key Properties:**
 - Every Recursive Language is also Recursively Enumerable.
 - A language L is Recursive if and only if both L and its complement $\bar{L}$ are Recursively Enumerable.
 - RE languages are closed under union, intersection, concatenation, Kleene star, homomorphism, inverse homomorphism. They are NOT closed under complement.
 - Recursive languages are closed under union, intersection, complement, concatenation, Kleene star, homomorphism, inverse homomorphism.
- **Common Pitfalls:** Confusing the definitions of "halts on all inputs" (decidable) vs. "halts and accepts" (recognizable); misinterpreting closure properties.
- **Problem-Solving Techniques:** Using closure properties to classify languages; applying reduction to prove undecidability/non-recognizability.

Decidability

Decidability refers to whether an algorithm (a Turing Machine that always halts) exists to solve a given problem. A problem is decidable if there's an algorithm that can always give a "yes" or "no" answer in finite time for any instance of the problem.

- **Definition:** A problem is decidable if the language representing all "yes" instances of the problem is a Recursive Language.
- **Key Properties (Examples of Decidable/Undecidable Problems):**
 - **Decidable:**
 - Membership problem for Regular Languages (Is $w \in L(FA)?$).
 - Emptiness problem for Regular Languages (Is $L(FA) = \emptyset?$).
 - Equivalence problem for Regular Languages (Is $L(FA_1) = L(FA_2)?$).
 - Membership problem for CFLs (Is $w \in L(CFG)?$).
 - Emptiness problem for CFLs (Is $L(CFG) = \emptyset?$).
 - **Undecidable:**
 - Halting Problem for TMs (Does TM M halt on input $w?$).
 - Equivalence problem for CFLs (Is $L(CFG_1) = L(CFG_2)?$).
 - Ambiguity problem for CFGs (Is CFG G ambiguous?).
 - Membership problem for RE languages (Is $w \in L(TM)?$ - this is RE, but not Recursive).
 - Emptiness problem for TMs (Is $L(TM) = \emptyset?$).
 - Post Correspondence Problem (PCP).
- **Common Pitfalls:** Assuming all problems are decidable; misremembering the decidability status of common problems.
- **Problem-Solving Techniques:**
 - **Reduction:** The primary method to prove a problem is undecidable.
 - Understanding Rice's Theorem (all non-trivial properties of RE languages are undecidable).

Reduction

Reduction is a powerful technique used to prove the undecidability of problems. If problem A can be "reduced" to problem B , it means that an algorithm for solving B could be used to solve A . Therefore, if A is known to be undecidable, B must also be undecidable.

- **Definition:** A problem A is reducible to problem B (denoted $A \leq_m B$ for many-one reduction) if there exists a computable function f that transforms every instance x of problem A into an instance $f(x)$ of problem B such that x is a "yes" instance of A if and only if $f(x)$ is a "yes" instance of B .
- **Key Properties:**
 - If $A \leq_m B$ and A is undecidable, then B is undecidable.
 - If $A \leq_m B$ and B is decidable, then A is decidable.
 - If $A \leq_m B$ and A is non-RE, then B is non-RE.
 - If $A \leq_m B$ and B is RE, then A is RE.
- **Common Pitfalls:** Reducing in the wrong direction (e.g., trying to reduce an undecidable problem to a decidable one); constructing an incorrect transformation function f .
- **Problem-Solving Techniques:**
 - Identify a known undecidable problem (e.g., Halting Problem, Emptiness Problem for TMs) to reduce FROM.

- Construct a transformation function f that takes an instance of the known undecidable problem and converts it into an instance of the target problem.
- Prove that this transformation preserves the "yes/no" answer.

Countable Uncountable Set

These concepts from set theory are crucial for understanding the limits of computation, particularly why there exist languages that are not recursively enumerable. It relates to the cardinality of sets.

• Definition:

- Countable Set:** A set is countable if it is finite or if its elements can be put into a one-to-one correspondence with the set of natural numbers $\mathbb{N}$ (i.e., it can be enumerated). Examples: integers $\mathbb{Z}$, rational numbers $\mathbb{Q}$, set of all finite strings Σ^* .
- Uncountable Set:** A set is uncountable if it is infinite but cannot be put into a one-to-one correspondence with $\mathbb{N}$. Examples: real numbers $\mathbb{R}$, power set of $\mathbb{N}$, set of all languages over Σ .

• Key Properties:

- The set of all Turing Machines is countable.
- The set of all possible languages over any alphabet Σ is uncountable.
- Since there are more languages than Turing Machines, there must exist languages that cannot be recognized by any Turing Machine (i.e., non-RE languages).

• Common Pitfalls: Confusing finite sets with infinite countable sets; misapplying diagonalization.

• Problem-Solving Techniques:

- Diagonalization Argument:** A standard technique to prove a set is uncountable (e.g., Cantor's diagonalization for real numbers).
- Understanding the implications for computability theory.

Identify Class Language

This topic refers to the ability to correctly classify a given language within the Chomsky Hierarchy, which organizes formal languages by their generative power and the type of automaton that recognizes them.

• Chomsky Hierarchy (from least to most powerful):

- Type 3: Regular Languages (RL):** Recognized by Finite Automata (FA). Generated by Regular Grammars (RG).
- Type 2: Context-Free Languages (CFL):** Recognized by Pushdown Automata (PDA). Generated by Context-Free Grammars (CFG). (Deterministic CFLs (DCFLs) are a proper subset of CFLs, recognized by DPDAs).
- Type 1: Context-Sensitive Languages (CSL):** Recognized by Linear Bounded Automata (LBA). Generated by Context-Sensitive Grammars (CSG). (Less common in GATE).
- Type 0: Recursively Enumerable Languages (RE):** Recognized by Turing Machines (TM). Generated by Unrestricted Grammars. (Recursive Languages are a proper subset of RE languages, recognized by TMs that always halt).

• Relationships: $RL \subset DCFL \subset CFL \subset CSL \subset \text{Recursive} \subset RE$

• Common Pitfalls: Incorrectly applying pumping lemmas; misjudging the memory requirements of a language; confusing closure properties.

• Problem-Solving Techniques:

- Pumping Lemma:** First attempt to prove non-regularity or non-context-freeness.
- Closure Properties:** Use them to combine or transform languages and deduce their class.
- Machine Models:** Try to design the simplest machine (FA, PDA, TM) that can recognize the language.
- Grammar Rules:** Analyze the structure of production rules if a grammar is given.

Medium (Computational Models)

In the context of Theory of Computation, "Medium" refers to the abstract computational models or machines used to define and study computation. These models, such as Finite Automata, Pushdown Automata, and Turing Machines, represent different levels of computational power and memory capabilities.

• Definition: Computational models are abstract machines that define the capabilities and limitations of algorithms. Each model specifies a set of operations and memory structures it can use.

• Key Models and Their Power:

- Finite Automata (FA):** No auxiliary memory. Recognizes Regular Languages. Simplest model.
- Pushdown Automata (PDA):** Finite control + a stack (LIFO memory). Recognizes Context-Free Languages. More powerful than FA.

- **Turing Machine (TM):** Finite control + an infinite tape (random access memory). Recognizes Recursively Enumerable Languages. Most powerful model, equivalent to any real-world computer.
- **Key Properties:**
 - The hierarchy of computational power ($FA < PDA < TM$) directly corresponds to the Chomsky Hierarchy of languages.
 - Each model defines the class of languages it can recognize (its "computational power").
- **Common Pitfalls:** Confusing the memory capabilities of different models; overestimating or underestimating the power of a specific model.
- **Problem-Solving Techniques:**
 - Analyze the memory requirements of a language to determine the minimum computational model needed to recognize it.
 - Understand how each model's memory structure (or lack thereof) limits its ability to recognize certain patterns (e.g., FA cannot count arbitrary numbers of symbols).

Quick Formula Reference

- **DFA:** $M = (Q, \Sigma, \delta, q_0, F)$
- **NFA:** $M = (Q, \Sigma, \delta, q_0, F)$, where $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$
- **Regular Expression Operations:** Union ($R_1 + R_2$), Concatenation ($R_1 R_2$), Kleene Star (R^*)
- **Arden's Theorem:** If $R = Q + RP$ then $R = QP^*$.
- **Regular Grammar (Right-linear):** $A \rightarrow aB$ or $A \rightarrow a$ or $A \rightarrow \epsilon$
- **Pumping Lemma for Regular Languages:** For $s = xyz \in L$ with $|s| \geq p$, $|y| > 0$, $|xy| \leq p$, $\forall i \geq 0$, $xy^i z \in L$.
- **Context-Free Grammar (CFG):** $G = (V, \Sigma, P, S)$, with productions $A \rightarrow \alpha$.
- **Chomsky Normal Form (CNF):** $A \rightarrow BC$ or $A \rightarrow a$.
- **Greibach Normal Form (GNF):** $A \rightarrow a\alpha$.
- **Pushdown Automata (PDA):** $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, with transitions $\delta(q, a, X) = \{(q', Y_1 Y_2 \dots Y_k)\}$
- **Pumping Lemma for Context-Free Languages:** For $s = uvwxy \in L$ with $|s| \geq p$, $|vx| > 0$, $|vwx| \leq p$, $\forall i \geq 0$, $uv^iwx^iy \in L$.
- **Turing Machine (TM):** $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, with transitions $\delta(q, X) = (q', Y, D)$.
- **Chomsky Hierarchy:** Regular $\subset$ DCFL $\subset$ CFL $\subset$ CSL $\subset$ Recursive $\subset$ RE.
- **Closure Properties (Summary):**

Operation	RL	CFL	Recursive	RE
Union	Y	Y	Y	Y
Intersection	Y	N	Y	Y
Complement	Y	N	Y	N
Concatenation	Y	Y	Y	Y
Kleene Star	Y	Y	Y	Y
Reversal	Y	Y	Y	Y

Important Tips for GATE

1. **Master the Chomsky Hierarchy:** Understand the relationships between language classes (Regular, CFL, Recursive, RE) and their corresponding machine models (FA, PDA, TM). This is fundamental for classifying languages and solving many conceptual questions.
2. **Memorize Closure Properties:** Create a table and commit to memory which language classes are closed under which operations (union, intersection, complement, concatenation, Kleene star, reversal). This is a frequent source of direct questions.
3. **Practice Pumping Lemma Proofs:** The Pumping Lemma for both Regular and Context-Free Languages is a crucial tool. Practice choosing the "hard" string and demonstrating the contradiction for various non-regular/non-CFLs. Remember, it only proves non-membership.
4. **Understand NFA to DFA Conversion and Minimization:** Be proficient in subset construction for NFA to DFA conversion and the table-filling algorithm for DFA minimization. These are common problem-solving tasks.
5. **Distinguish Decidability vs. Recognizability:** Clearly understand the difference between Recursive (decidable) and Recursively Enumerable (recognizable) languages, especially regarding the halting behavior of Turing Machines. Know the decidability status of common problems (Halting Problem, Emptiness for TMs, Equivalence for CFLs, etc.).
6. **Focus on Design Principles:** While complex designs are rare, understand the core principles of designing simple FAs, PDAs, and TMs for basic languages. Pay attention to how memory (or lack thereof) is utilized.
7. **Pay Attention to Formal Definitions:** GATE questions often test your understanding of the precise formal

definitions of automata, grammars, and languages. Small details like epsilon transitions, initial stack symbols, or blank tape symbols can be critical.

8. **Time Management for Tricky Questions:** Some ToC problems, especially those involving complex reductions or intricate Pumping Lemma applications, can be time-consuming. Practice identifying these quickly and decide whether to attempt them or mark for review.

6.1

Closure Property (10)

 **Practice Test:** Test (12Q)

6.1.1 Closure Property: GATE CSE 1989 | Question: 3-ii



Context-free languages and regular languages are both closed under the operation(s) of :

- A. Union B. Intersection C. Concatenation D. Complementation

gate1989 easy theory-of-computation closure-property multiple-selects

Answer key

6.1.2 Closure Property: GATE CSE 1992 | Question: 16



Which of the following three statements are true? Prove your answer.

- The union of two recursive languages is recursive.
- The language $\{0^n \mid n \text{ is a prime}\}$ is not regular.
- Regular languages are closed under infinite union.

gate1992 theory-of-computation normal closure-property proof descriptive

Answer key

6.1.3 Closure Property: GATE CSE 2002 | Question: 2.14



Which of the following is true?

- A. The complement of a recursive language is recursive.
B. The complement of a recursively enumerable language is recursively enumerable.
C. The complement of a recursive language is either recursive or recursively enumerable.
D. The complement of a context-free language is context-free.

gatecse-2002 theory-of-computation easy closure-property

Answer key

6.1.4 Closure Property: GATE CSE 2013 | Question: 17



Which of the following statements is/are **FALSE**?

- For every non-deterministic Turing machine, there exists an equivalent deterministic Turing machine.
- Turing recognizable languages are closed under union and complementation.
- Turing decidable languages are closed under intersection and complementation.
- Turing recognizable languages are closed under union and intersection.

- A. 1 and 4 only B. 1 and 3 only
C. 2 only D. 3 only

gatecse-2013 theory-of-computation normal closure-property

Answer key

6.1.5 Closure Property: GATE CSE 2016 | Set 2 | Question: 18



Consider the following types of languages: L_1 : Regular, L_2 : Context-free, L_3 : Recursive, L_4 : Recursively enumerable. Which of the following is/are **TRUE**?

- $\overline{L_3} \cup L_4$ is recursively enumerable.

- II. $\overline{L_2} \cup L_3$ is recursive.
 III. $L_1^* \cap L_2$ is context-free.
 IV. $L_1 \cup \overline{L_2}$ is context-free.

- A. I only. B. I and III only. C. I and IV only. D. I, II and III only.

gatecse-2016-set2 theory-of-computation regular-language context-free-language closure-property normal

Answer key

6.1.6 Closure Property: GATE CSE 2017 | Set 2 | Question: 04



Let L_1, L_2 be any two context-free languages and R be any regular language. Then which of the following is/are CORRECT?

- I. $L_1 \cup L_2$ is context-free
 II. $\overline{L_1}$ is context-free
 III. $L_1 - R$ is context-free
 IV. $L_1 \cap L_2$ is context-free

- A. I, II and IV only. B. I and III only. C. II and IV only. D. I only

gatecse-2017-set2 theory-of-computation closure-property

Answer key

6.1.7 Closure Property: GATE CSE 2018 | Question: 7



The set of all recursively enumerable languages is:

- A. closed under complementation. B. closed under intersection.
 C. a subset of the set of all recursive languages. D. an uncountable set.

gatecse-2018 theory-of-computation closure-property easy one-mark

Answer key

6.1.8 Closure Property: GATE CSE 2025 | Set 2 | Question: 20



Consider the two lists List I and List II given below:

List I	List II
Context-free languages	Closed under union
Recursive languages	Not closed under complementation
Regular languages	Closed under intersection

For matching of items in List I with those in List II, which of the following option(s) is/are CORRECT?

- A. (i) - (a), (ii) - (b), and (iii) - (c). B. (i) - (b), (ii) - (a), and (iii) - (c).
 C. (i) - (b), (ii) - (c), and (iii) - (a). D. (i) - (a), (ii) - (c), and (iii) - (b).

gatecse2025-set2 theory-of-computation closure-property match-the-following multiple-selects easy one-mark

Answer key

6.1.9 Closure Property: GATE CSE 2026 | Set 1 | Question: 41



Let L_1 and L_2 be two languages over a finite alphabet, such that $L_1 \cap L_2$ and L_2 are regular languages.

Which of the following statements is/are always true?

- A. L_1 is regular. B. $L_1 \cup L_2$ is regular.
 C. $\overline{L_2}$ is context-free. D. L_1 is context-free.

gatecse-2026-set1 theory-of-computation closure-property two-marks multiple-selects

6.1.10 Closure Property: GATE IT 2006 | Question: 32



Let L be a context-free language and M a regular language. Then the language $L \cap M$ is

- A. always regular
- B. never regular
- C. always a deterministic context-free language
- D. always a context-free language

gateit-2006 theory-of-computation closure-property easy

6.2

Context Free Grammar (2)

Practice Tests: Test 1 (11Q) Weekly Quiz 10 (10Q) Weekly Quiz 4 (10Q)

6.2.1 Context Free Grammar: GATE CSE 2025 | Set 1 | Question: 9



Consider the following context-free grammar G , where S , A , and B are the variables (non-terminals), a and b are the terminal symbols, S is the start variable, and the rules of G are described as:

$$S \rightarrow aaB \mid Abb$$

$$A \rightarrow a \mid aA$$

$$B \rightarrow b \mid bB$$

Which ONE of the languages $L(G)$ is accepted by G ?

- A. $L(G) = \{a^2b^n \mid n \geq 1\} \cup \{a^n b^2 \mid n \geq 1\}$
- B. $L(G) = \{a^n b^{2n} \mid n \geq 1\} \cup \{a^{2n} b^n \mid n \geq 1\}$
- C. $L(G) = \{a^n b^n \mid n \geq 1\}$
- D. $L(G) = \{a^{2n} b^{2n} \mid n \geq 1\}$

gatecse2025-set1 theory-of-computation context-free-grammar easy one-mark

6.2.2 Context Free Grammar: GATE CSE 2026 | Set 1 | Question: 15



Consider the following grammar where S is the start symbol, and a and b are terminal symbols.

$$S \rightarrow aSbS \mid bS \mid \epsilon$$

Which of the following statements is/are true?

- A. The grammar is ambiguous
- B. The string abb has two distinct derivations in this grammar
- C. The string $abab$ has only one rightmost derivation
- D. The language generated by the grammar is undecidable

gatecse-2026-set1 theory-of-computation context-free-grammar multiple-selects one-mark

6.3

Context Free Language (35)

Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (14Q)

6.3.1 Context Free Language: GATE CSE 1987 | Question: 2k



State whether the following statements are TRUE or FALSE:

The intersection of two CFL's is also a CFL.

gate1987 theory-of-computation context-free-language true-false

Answer key

6.3.2 Context Free Language: GATE CSE 1992 | Question: 02,xix



Context-free languages are:

- A. closed under union
- B. closed under complementation
- C. closed under intersection
- D. closed under Kleene closure

gate1992 context-free-language theory-of-computation normal multiple-selects

Answer key

6.3.3 Context Free Language: GATE CSE 1995 | Question: 2.20



Which of the following definitions below generate the same language as L , where $L = \{x^n y^n \text{ such that } n \geq 1\}$?

- I. $E \rightarrow xEy \mid xy$
- II. $xy \mid (x^+ xyy^+)$
- III. $x^+ y^+$

- A. I only
- B. I and II
- C. II and III
- D. II only

gate1995 theory-of-computation easy context-free-language

Answer key

6.3.4 Context Free Language: GATE CSE 1996 | Question: 2.8



If L_1 and L_2 are context free languages and R a regular set, one of the languages below is not necessarily a context free language. Which one?

- A. $L_1 \cdot L_2$
- B. $L_1 \cap L_2$
- C. $L_1 \cap R$
- D. $L_1 \cup L_2$

gate1996 theory-of-computation context-free-language easy

Answer key

6.3.5 Context Free Language: GATE CSE 1996 | Question: 2.9



Define a context free languages $L \in \{0, 1\}^*$, $\text{init}(L) = \{u \mid uv \in L \text{ for some } v \text{ in } \{0, 1\}^*\}$ (in other words, $\text{init}(L)$ is the set of prefixes of L)

Let $L = \{w \mid w \text{ is nonempty and has an equal number of 0's and 1's}\}$

Then $\text{init}(L)$ is:

- A. the set of all binary strings with unequal number of 0's and 1's
- B. the set of all binary strings including null string
- C. the set of all binary strings with exactly one more 0 than the number of 1's or one more 1 than the number of 0's
- D. None of the above

gate1996 theory-of-computation context-free-language normal

Answer key

6.3.6 Context Free Language: GATE CSE 1999 | Question: 1.5



Context-free languages are closed under:

- A. Union, intersection
- B. Union, Kleene closure
- C. Intersection, complement
- D. Complement, Kleene closure

Answer key

6.3.7 Context Free Language: GATE CSE 1999 | Question: 7

Show that the language

$$L = \{xcx \mid x \in \{0,1\}^* \text{ and } c \text{ is a terminal symbol}\}$$

is not context free. c is not 0 or 1.

Answer key

6.3.8 Context Free Language: GATE CSE 2000 | Question: 7

A. Construct a minimal finite state machine that accepts the language, over $\{0,1\}$, of all strings that contain neither the substring 00 nor the substring 11.

B. Consider the grammar

- $S \rightarrow aSAb$
- $S \rightarrow \epsilon$
- $A \rightarrow bA$
- $A \rightarrow \epsilon$

where S, A are non-terminal symbols with S being the start symbol; a, b are terminal symbols and ϵ is the empty string. This grammar generates strings of the form $a^i b^j$ for some $i, j \geq 0$, where i and j satisfy some condition. What is the condition on the values of i and j ?

Answer key

6.3.9 Context Free Language: GATE CSE 2001 | Question: 1.5

Which of the following statements is true?

- A. If a language is context free it can always be accepted by a deterministic push-down automaton
- B. The union of two context free languages is context free
- C. The intersection of two context free languages is a context free
- D. The complement of a context free language is a context free

Answer key

6.3.10 Context Free Language: GATE CSE 2003 | Question: 51

Let $G = (\{S\}, \{a, b\}, R, S)$ be a context free grammar where the rule set R is $S \rightarrow aSb \mid SS \mid \epsilon$

Which of the following statements is true?

- A. G is not ambiguous
- B. There exist $x, y \in L(G)$ such that $xy \notin L(G)$
- C. There is a deterministic pushdown automaton that accepts $L(G)$
- D. We can find a deterministic finite state automaton that accepts $L(G)$

Answer key

6.3.11 Context Free Language: GATE CSE 2005 | Question: 57

Consider the languages:

- $L_1 = \{ww^R \mid w \in \{0,1\}^*\}$
- $L_2 = \{w\#w^R \mid w \in \{0,1\}^*\}$, where $\#$ is a special symbol
- $L_3 = \{ww \mid w \in \{0,1\}^*\}$

Which one of the following is TRUE?

- A. L_1 is a deterministic CFL
- B. L_2 is a deterministic CFL
- C. L_3 is a CFL, but not a deterministic CFL
- D. L_3 is a deterministic CFL

gatecse-2005 theory-of-computation context-free-language easy

Answer key

6.3.12 Context Free Language: GATE CSE 2006 | Question: 19



Let

$$L_1 = \{0^{n+m}1^n0^m \mid n, m \geq 0\},$$

$$L_2 = \{0^{n+m}1^{n+m}0^m \mid n, m \geq 0\} \text{ and}$$

$$L_3 = \{0^{n+m}1^{n+m}0^{n+m} \mid n, m \geq 0\}.$$

Which of these languages are NOT context free?

- A. L_1 only
- B. L_3 only
- C. L_1 and L_2
- D. L_2 and L_3

gatecse-2006 theory-of-computation context-free-language normal

Answer key

6.3.13 Context Free Language: GATE CSE 2009 | Question: 12, ISRO2016-37



$$S \rightarrow aSa \mid bSb \mid a \mid b$$

The language generated by the above grammar over the alphabet $\{a, b\}$ is the set of:

- A. all palindromes
- B. all odd length palindromes
- C. strings that begin and end with the same symbol
- D. all even length palindromes

gatecse-2009 theory-of-computation context-free-language easy isro2016

Answer key

6.3.14 Context Free Language: GATE CSE 2015 | Set 3 | Question: 32



Which of the following languages are context-free?

$$L_1 : \{a^m b^n a^n b^m \mid m, n \geq 1\}$$

$$L_2 : \{a^m b^n a^m b^n \mid m, n \geq 1\}$$

$$L_3 : \{a^m b^n \mid m = 2n + 1\}$$

- A. L_1 and L_2 only
- B. L_1 and L_3 only
- C. L_2 and L_3 only
- D. L_3 only

gatecse-2015-set3 theory-of-computation context-free-language normal

Answer key

6.3.15 Context Free Language: GATE CSE 2016 | Set 1 | Question: 16



Which of the following languages is generated by the given grammar?

$$S \rightarrow aS \mid bS \mid \epsilon$$

- A. $\{a^n b^m \mid n, m \geq 0\}$
- B. $\{w \in \{a, b\}^* \mid w \text{ has equal number of a's and b's}\}$
- C. $\{a^n \mid n \geq 0\} \cup \{b^n \mid n \geq 0\} \cup \{a^n b^n \mid n \geq 0\}$

D. $\{a, b\}^*$

gatecse-2016-set1 theory-of-computation context-free-language normal

Answer key

6.3.16 Context Free Language: GATE CSE 2016 | Set 1 | Question: 42



Consider the following context-free grammars:

$$G_1 : S \rightarrow aS \mid B, B \rightarrow b \mid bB$$

$$G_2 : S \rightarrow aA \mid bB, A \rightarrow aA \mid B \mid \varepsilon, B \rightarrow bB \mid \varepsilon$$

Which one of the following pairs of languages is generated by G_1 and G_2 , respectively?

- A. $\{a^m b^n \mid m > 0 \text{ or } n > 0\}$ and $\{a^m b^n \mid m > 0 \text{ and } n > 0\}$
- B. $\{a^m b^n \mid m > 0 \text{ and } n > 0\}$ and $\{a^m b^n \mid m > 0 \text{ or } n \geq 0\}$
- C. $\{a^m b^n \mid m \geq 0 \text{ or } n > 0\}$ and $\{a^m b^n \mid m > 0 \text{ and } n > 0\}$
- D. $\{a^m b^n \mid m \geq 0 \text{ and } n > 0\}$ and $\{a^m b^n \mid m > 0 \text{ or } n > 0\}$

gatecse-2016-set1 theory-of-computation context-free-language normal

Answer key

6.3.17 Context Free Language: GATE CSE 2016 | Set 2 | Question: 43



Consider the following languages:

- $L_1 = \{a^n b^m c^{n+m} : m, n \geq 1\}$
- $L_2 = \{a^n b^n c^{2n} : n \geq 1\}$

Which one of the following is TRUE?

- A. Both L_1 and L_2 are context-free.
- B. L_1 is context-free while L_2 is not context-free.
- C. L_2 is context-free while L_1 is not context-free.
- D. Neither L_1 nor L_2 is context-free.

gatecse-2016-set2 theory-of-computation context-free-language normal

Answer key

6.3.18 Context Free Language: GATE CSE 2017 | Set 1 | Question: 10



Consider the following context-free grammar over the alphabet $\Sigma = \{a, b, c\}$ with S as the start symbol:

$$S \rightarrow abScT \mid abcT$$

$$T \rightarrow bT \mid b$$

Which one of the following represents the language generated by the above grammar?

- A. $\{(ab)^n (cb)^n \mid n \geq 1\}$
- B. $\{(ab)^n cb^{m_1} cb^{m_2} \dots cb^{m_n} \mid n, m_1, m_2, \dots, m_n \geq 1\}$
- C. $\{(ab)^n (cb^m)^n \mid m, n \geq 1\}$
- D. $\{(ab)^n (cb^n)^m \mid m, n \geq 1\}$

gatecse-2017-set1 theory-of-computation context-free-language normal

Answer key

6.3.19 Context Free Language: GATE CSE 2017 | Set 1 | Question: 34



If G is a grammar with productions

$$S \rightarrow SaS \mid aSb \mid bSa \mid SS \mid \epsilon$$

where S is the start variable, then which one of the following strings is not generated by G ?

- A. $abab$ B. $aaab$ C. $abbaa$ D. $babba$

gatecse-2017-set1 theory-of-computation context-free-language normal

Answer key

6.3.20 Context Free Language: GATE CSE 2017 | Set 1 | Question: 38



Consider the following languages over the alphabet $\Sigma = \{a, b, c\}$. Let $L_1 = \{a^n b^n c^n \mid m, n \geq 0\}$ and $L_2 = \{a^m b^n c^n \mid m, n \geq 0\}$.

Which of the following are context-free languages?

- I. $L_1 \cup L_2$
II. $L_1 \cap L_2$

- A. I only B. II only C. I and II D. Neither I nor II

gatecse-2017-set1 theory-of-computation context-free-language normal

Answer key

6.3.21 Context Free Language: GATE CSE 2017 | Set 2 | Question: 16



Identify the language generated by the following grammar, where S is the start variable.

- $S \rightarrow XY$
- $X \rightarrow aX \mid a$
- $Y \rightarrow aYb \mid \epsilon$

- A. $\{a^m b^n \mid m \geq n, n > 0\}$ B. $\{a^m b^n \mid m \geq n, n \geq 0\}$
C. $\{a^m b^n \mid m > n, n \geq 0\}$ D. $\{a^m b^n \mid m > n, n > 0\}$

gatecse-2017-set2 theory-of-computation context-free-language

Answer key

6.3.22 Context Free Language: GATE CSE 2019 | Question: 31



Which one of the following languages over $\Sigma = \{a, b\}$ is NOT context-free?

- A. $\{ww^R \mid w \in \{a, b\}^*\}$
B. $\{wa^n b^n w^R \mid w \in \{a, b\}^*, n \geq 0\}$
C. $\{wa^n w^R b^n \mid w \in \{a, b\}^*, n \geq 0\}$
D. $\{a^n b^i \mid i \in \{n, 3n, 5n\}, n \geq 0\}$

gatecse-2019 theory-of-computation context-free-language two-marks

Answer key

6.3.23 Context Free Language: GATE CSE 2021 | Set 1 | Question: 1



Suppose that L_1 is a regular language and L_2 is a context-free language. Which one of the following languages is NOT necessarily context-free?

- A. $L_1 \cap L_2$ B. $L_1 \cdot L_2$ C. $L_1 - L_2$ D. $L_1 \cup L_2$

gatecse-2021-set1 context-free-language theory-of-computation one-mark

Answer key

6.3.24 Context Free Language: GATE CSE 2021 | Set 2 | Question: 41



For a string w , we define w^R to be the reverse of w . For example, if $w = 01101$ then $w^R = 10110$.

Which of the following languages is/are context-free?

- A. $\{wxw^R x^R \mid w, x \in \{0, 1\}^*\}$ B. $\{ww^R xx^R \mid w, x \in \{0, 1\}^*\}$
C. $\{wxw^R \mid w, x \in \{0, 1\}^*\}$ D. $\{wxx^R w^R \mid w, x \in \{0, 1\}^*\}$

gatecse-2021-set2 multiple-selects theory-of-computation context-free-language two-marks

Answer key

6.3.25 Context Free Language: GATE CSE 2022 | Question: 38



Consider the following languages:

- $L_1 = \{ww \mid w \in \{a, b\}^*\}$
- $L_2 = \{a^n b^n c^m \mid m, n \geq 0\}$
- $L_3 = \{a^m b^n c^n \mid m, n \geq 0\}$

Which of the following statements is/are FALSE?

- A. L_1 is not context-free but L_2 and L_3 are deterministic context-free.
B. Neither L_1 nor L_2 is context-free.
C. L_2, L_3 and $L_2 \cap L_3$ all are context-free.
D. Neither L_1 nor its complement is context-free.

gatecse-2022 theory-of-computation context-free-language multiple-selects two-marks

Answer key

6.3.26 Context Free Language: GATE CSE 2023 | Question: 29



Consider the context-free grammar G below

$$\begin{aligned} S &\rightarrow aSb \mid X \\ X &\rightarrow aX \mid Xb \mid a \mid b, \end{aligned}$$

where S and X are non-terminals, and a and b are terminal symbols. The starting non-terminal is S .

Which one of the following statements is CORRECT?

- A. The language generated by G is $(a + b)^*$
B. The language generated by G is $a^*(a + b)b^*$
C. The language generated by G is $a^*b^*(a + b)$
D. The language generated by G is not a regular language

gatecse-2023 theory-of-computation context-free-language two-marks

Answer key

6.3.27 Context Free Language: GATE CSE 2025 | Set 1 | Question: 35



Consider the following two languages over the alphabet $\{a, b, c\}$, where m and n are natural numbers.

$$\begin{aligned} L_1 &= \{a^m b^m c^{m+n} \mid m, n \geq 1\} \\ L_2 &= \{a^m b^n c^{m+n} \mid m, n \geq 1\} \end{aligned}$$

Which ONE of the following statements is CORRECT?

- A. Both L_1 and L_2 are context-free languages.
B. L_1 is a context-free language but L_2 is not a context-free language.
C. L_1 is not a context-free language but L_2 is a context-free language.
D. Neither L_1 nor L_2 are context-free languages.

gatecse2025-set1 theory-of-computation context-free-language two-marks

6.3.28 Context Free Language: GATE CSE 2026 | Set 1 | Question: 42



Consider the following context-free grammar G

$$\begin{aligned} S &\rightarrow abaABAbba \\ A &\rightarrow aaBBAb \mid bBabaa \\ B &\rightarrow aBb \mid ab \end{aligned}$$

In the above grammar, S is the start symbol, a and b are terminal symbols, and A and B are non-terminal symbols.

Let $L(G)$ be the language generated by the grammar G . For a string $s \in L(G)$, let $n_1(s)$ be the number of a 's in s and $n_2(s)$ be the number of b 's in s .

Which of the following statements is/are true?

- A. There is a string $s \in L(G)$ such that $n_1(s) < n_2(s)$
- B. For every string $s \in L(G)$, $n_1(s) > n_2(s)$
- C. There is a string $s \in L(G)$ such that $n_1(s) > 2n_2(s)$
- D. For every string $s \in L(G)$, $n_1(s) \leq 2n_2(s)$

gatescse-2026-set1 theory-of-computation context-free-language two-marks multiple-selects

6.3.29 Context Free Language: GATE CSE 2026 | Set 2 | Question: 38



Let $\Sigma = \{a, b, c, d\}$ and let $L = \{a^i b^j c^k d^\ell \mid i, j, k, \ell \geq 0\}$.

Which of the following constraints ensure(s) that the language L is context-free?

- A. $i + k = j + \ell$
- B. $i = k$ and $j = \ell$
- C. $i = \ell$ and $j = k$
- D. $i + j = k + \ell$

gatescse-2026-set2 theory-of-computation context-free-language multiple-selects two-marks

6.3.30 Context Free Language: GATE IT 2006 | Question: 34



In the context-free grammar below, S is the start symbol, a and b are terminals, and ϵ denotes the empty string.

$$\begin{aligned} &S \rightarrow aSAb \mid \epsilon \\ &A \rightarrow bA \mid \epsilon \end{aligned}$$

The grammar generates the language

- A. $((a+b)^*b)$
- B. $\{a^m b^n \mid m \leq n\}$
- C. $\{a^m b^n \mid m = n\}$
- D. $a^* b^*$

gateit-2006 theory-of-computation context-free-language normal

6.3.31 Context Free Language: GATE IT 2006 | Question: 4



In the context-free grammar below, S is the start symbol, a and b are terminals, and ϵ denotes the empty string.

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid \epsilon$$

Which of the following strings is NOT generated by the grammar?

A. aaaaB. babaC. abbaD. babaaabab

gateit-2006

theory-of-computation

context-free-language

easy

Answer key

6.3.32 Context Free Language: GATE IT 2007 | Question: 46



The two grammars given below generate a language over the alphabet $\{x, y, z\}$

- $G_1 : S \rightarrow x \mid z \mid xS \mid zS \mid yB$
 $B \rightarrow y \mid z \mid yB \mid zB$
- $G_2 : S \rightarrow y \mid z \mid yS \mid zS \mid xB$
 $B \rightarrow y \mid yS$

Which one of the following choices describes the properties satisfied by the strings in these languages?

- A. G_1 : No y appears before any x
 G_2 : Every x is followed by at least one y
- B. G_1 : No y appears before any x
 G_2 : No x appears before any y
- C. G_1 : No y appears after any x
 G_2 : Every x is followed by at least one y
- D. G_1 : No y appears after any x
 G_2 : Every y is followed by at least one x

gateit-2007

theory-of-computation

normal

context-free-language

Answer key

6.3.33 Context Free Language: GATE IT 2007 | Question: 48



Consider the grammar given below:

$$\begin{aligned} S &\rightarrow xB \mid yA \\ A &\rightarrow x \mid xS \mid yAA \\ B &\rightarrow y \mid yS \mid xBB \end{aligned}$$

Consider the following strings.

- i. xyyx
- ii. xyyyxy
- iii. xyxy
- iv. yxxxy
- v. yxx
- vi. xyx

Which of the above strings are generated by the grammar?

- A. i, ii and iii
- B. ii, v and vi
- C. ii, iii and iv
- D. i, iii and iv

gateit-2007

theory-of-computation

context-free-language

normal

Answer key

6.3.34 Context Free Language: GATE IT 2007 | Question: 49



Consider the following grammars. Names representing terminals have been specified in capital letters.

$\begin{array}{l} \text{G1 : } \& \text{\texttt{\text{stmt}}} \& \text{\texttt{\text{WHILE (expr) stmt}}} \end{array}$

Which one of the following statements is true?

- A. G_1 is context-free but not regular and G_2 is regular
- B. G_2 is context-free but not regular and G_1 is regular

- C. Both G_1 and G_2 are regular
 D. Both G_1 and G_2 are context-free but neither of them is regular

gateit-2007 theory-of-computation context-free-language normal

Answer key

6.3.35 Context Free Language: GATE IT 2008 | Question: 34



Consider a CFG with the following productions.

$$\begin{aligned} S &\rightarrow AA \mid B \\ A &\rightarrow 0A \mid A0 \mid 1 \\ B &\rightarrow 0B00 \mid 1 \end{aligned}$$

S is the start symbol, A and B are non-terminals and 0 and 1 are the terminals. The language generated by this grammar is:

- A. $\{0^n 10^{2n} \mid n \geq 1\}$
 B. $\{0^i 10^j 10^k \mid i, j, k \geq 0\} \cup \{0^n 10^{2n} \mid n \geq 0\}$
 C. $\{0^i 10^j \mid i, j \geq 0\} \cup \{0^n 10^{2n} \mid n \geq 0\}$
 D. The set of all strings over $\{0, 1\}$ containing at least two 0's

gateit-2008 theory-of-computation context-free-language normal

Answer key

6.4 Countable Uncountable Set (3)

Practice Test: Test 1 (9Q)

6.4.1 Countable Uncountable Set: GATE CSE 1997 | Question: 3.4



Given $\Sigma = \{a, b\}$, which one of the following sets is not countable?

- A. Set of all strings over Σ
 B. Set of all languages over Σ
 C. Set of all regular languages over Σ
 D. Set of all languages over Σ accepted by Turing machines

gate1997 theory-of-computation normal countable-uncountable-set

Answer key

6.4.2 Countable Uncountable Set: GATE CSE 2014 | Set 3 | Question: 16



Let Σ be a finite non-empty alphabet and let 2^{Σ^*} be the power set of Σ^* . Which one of the following is TRUE?

- A. Both 2^{Σ^*} and Σ^* are countable
 B. 2^{Σ^*} is countable and Σ^* is uncountable
 C. 2^{Σ^*} is uncountable and Σ^* is countable
 D. Both 2^{Σ^*} and Σ^* are uncountable

gatecse-2014-set3 theory-of-computation normal countable-uncountable-set

Answer key

6.4.3 Countable Uncountable Set: GATE CSE 2019 | Question: 34



Consider the following sets:

S1: Set of all recursively enumerable languages over the alphabet $\{0, 1\}$

S2: Set of all syntactically valid C programs
 S3: Set of all languages over the alphabet $\{0, 1\}$
 S4: Set of all non-regular languages over the alphabet $\{0, 1\}$
 Which of the above sets are uncountable?

- A. S1 and S2 B. S3 and S4 C. S2 and S3 D. S1 and S4

gatecse-2019 theory-of-computation countable-uncountable-set two-marks

Answer key

6.5 Decidability (30)

Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (8Q) Weekly Quiz 6 (10Q)

6.5.1 Decidability: GATE CSE 1987 | Question: 2l

State whether the following statement are TRUE or FALSE.

A is recursive if both A and its complement are accepted by Turing machines.

gate1987 theory-of-computation turing-machine decidability true-false

Answer key

6.5.2 Decidability: GATE CSE 1987 | Question: 2m

State whether the following statements are TRUE or FALSE:

The problem as to whether a Turing machine M accepts input w is undecidable.

gate1987 theory-of-computation turing-machine decidability true-false

Answer key

6.5.3 Decidability: GATE CSE 1988 | Question: 2viii

State the halting problem of the Turing machine.

gate1988 theory-of-computation descriptive decidability turing-machine

Answer key

6.5.4 Decidability: GATE CSE 1989 | Question: 3-iii

Which of the following problems are undecidable?

- A. Membership problem in context-free languages.
 B. Whether a given context-free language is regular.
 C. Whether a finite state automation halts on all inputs.
 D. Membership problem for type 0 languages.

gate1989 normal theory-of-computation decidability multiple-selects

Answer key

6.5.5 Decidability: GATE CSE 1990 | Question: 3-vii

It is undecidable whether:

- A. An arbitrary Turing machine halts after 100 steps.
 B. A Turing machine prints a specific letter.
 C. A Turing machine computes the products of two numbers
 D. None of the above.

gate1990 normal theory-of-computation decidability multiple-selects

Answer key

6.5.6 Decidability: GATE CSE 1995 | Question: 11



Let L be a language over Σ i.e., $L \subseteq \Sigma^*$. Suppose L satisfies the two conditions given below.

- L is in NP and
- For every n , there is exactly one string of length n that belongs to L .

Let L^c be the complement of L over Σ^* . Show that L^c is also in NP.

gate1995 theory-of-computation normal decidability proof descriptive

Answer key

6.5.7 Decidability: GATE CSE 1996 | Question: 1.9



Which of the following statements is false?

- The Halting Problem of Turing machines is undecidable
- Determining whether a context-free grammar is ambiguous is undecidable
- Given two arbitrary context-free grammars G_1 and G_2 it is undecidable whether $L(G_1) = L(G_2)$
- Given two regular grammars G_1 and G_2 it is undecidable whether $L(G_1) = L(G_2)$

gate1996 theory-of-computation decidability easy

Answer key

6.5.8 Decidability: GATE CSE 1997 | Question: 6.5



Which one of the following is not decidable?

- Given a Turing machine M , a string s and an integer k , M accepts s within k steps
- Equivalence of two given Turing machines
- Language accepted by a given finite state machine is not empty
- Language generated by a context free grammar is non-empty

gate1997 theory-of-computation decidability easy

Answer key

6.5.9 Decidability: GATE CSE 1999 | Question: 10



Suppose we have a function HALTS which when applied to any arbitrary function f and its arguments will say TRUE if function f terminates for those arguments and FALSE otherwise. Example: Given the following function definition.

FACTORIAL (N) = IF ($N=0$) THEN 1 ELSE $N \times \text{FACTORIAL}(N-1)$

Then HALTS (FACTORIAL, 4) = TRUE and HALTS (FACTORIAL, -5) = FALSE

Let us define the function FUNNY (f) = IF HALTS (f) THEN not (f) ELSE TRUE

- Show that FUNNY terminates for all functions f .
- use (a) to prove (by contradiction) that it is not possible to have a function like HALTS which for arbitrary functions and inputs says whether it will terminate on that input or not.

gate1999 theory-of-computation descriptive decidability

Answer key

6.5.10 Decidability: GATE CSE 2000 | Question: 2.9



Consider the following decision problems:

(P_1) : Does a given finite state machine accept a given string?

(P2) : Does a given context free grammar generate an infinite number of strings?

Which of the following statements is true?

- A. Both (P1) and (P2) are decidable
- B. Neither (P1) nor (P2) is decidable
- C. Only (P1) is decidable
- D. Only (P2) is decidable

gatecse-2000 theory-of-computation decidability normal

Answer key

6.5.11 Decidability: GATE CSE 2001 | Question: 2.7



Consider the following problem X .

Given a Turing machine M over the input alphabet Σ , any state q of M and a word $w \in \Sigma^*$, does the computation of M on w visit the state of q ?

Which of the following statements about X is correct?

- A. X is decidable
- B. X is undecidable but partially decidable
- C. X is undecidable and not even partially decidable
- D. X is not a decision problem

gatecse-2001 theory-of-computation decidability normal

Answer key

6.5.12 Decidability: GATE CSE 2001 | Question: 7



Let a decision problem X be defined as follows:

X : Given a Turing machine M over Σ and any word $w \in \Sigma$, does M loop forever on w ?

You may assume that the halting problem of Turing machine is undecidable but partially decidable.

- A. Show that X is undecidable
- B. Show that X is not even partially decidable

gatecse-2001 theory-of-computation decidability turing-machine easy descriptive

Answer key

6.5.13 Decidability: GATE CSE 2002 | Question: 14



The aim of the following question is to prove that the language $\{M \mid M \text{ is the code of the Turing Machine which, irrespective of the input, halts and outputs a } 1\}$ is undecidable. This is to be done by reducing from the language $\{M', x \mid M' \text{ halts on } x\}$, which is known to be undecidable. In parts (a) and (b) describe the 2 main steps in the construction of M . In part (c) describe the key property which relates the behaviour of M on its input w to the behaviour of M' on x .

- A. On input w , what is the first step that M must make?
- B. On input w , based on the outcome of the first step, what is the second step M must make?
- C. What key property relates the behaviour of M on w to the behaviour of M' on x ?

gatecse-2002 theory-of-computation decidability normal turing-machine descriptive difficult

Answer key

6.5.14 Decidability: GATE CSE 2003 | Question: 52



Consider two languages L_1 and L_2 each on the alphabet Σ . Let $f : \Sigma^* \rightarrow \Sigma^*$ be a polynomial time computable bijection such that $(\forall x)[x \in L_1 \text{ iff } f(x) \in L_2]$. Further, let f^{-1} be also polynomial time computable.

Which of the following **CANNOT** be true?

- A. $L_1 \in P$ and L_2 is finite
 B. $L_1 \in NP$ and $L_2 \in P$
 C. L_1 is undecidable and L_2 is decidable
 D. L_1 is recursively enumerable and L_2 is recursive

gatecse-2003 theory-of-computation difficult decidability

Answer key

6.5.15 Decidability: GATE CSE 2003 | Question: 53

A single tape Turing Machine M has two states q_0 and q_1 , of which q_0 is the starting state. The tape alphabet of M is $\{0, 1, B\}$ and its input alphabet is $\{0, 1\}$. The symbol B is the blank symbol used to indicate end of an input string. The transition function of M is described in the following table.

	0	1	B
q_0	$q_1, 1, R$	$q_1, 1, R$	$Halt$
q_1	$q_1, 1, R$	$q_0, 1, L$	q_0, B, L

The table is interpreted as illustrated below.

The entry $(q_1, 1, R)$ in row q_0 and column 1 signifies that if M is in state q_0 and reads 1 on the current tape square, then it writes 1 on the same tape square, moves its tape head one position to the right and transitions to state q_1 .

Which of the following statements is true about M ?

- A. M does not halt on any string in $(0+1)^+$
 B. M does not halt on any string in $(00+1)^*$
 C. M halts on all strings ending in a 0
 D. M halts on all strings ending in a 1

gatecse-2003 theory-of-computation decidability normal

Answer key

6.5.16 Decidability: GATE CSE 2007 | Question: 6

Which of the following problems is undecidable?

- A. Membership problem for CFGs
 B. Ambiguity problem for CFGs
 C. Finiteness problem for FSAs
 D. Equivalence problem for FSAs

gatecse-2007 theory-of-computation decidability normal

Answer key

6.5.17 Decidability: GATE CSE 2008 | Question: 10

Which of the following are decidable?

- I. Whether the intersection of two regular languages is infinite
 II. Whether a given context-free language is regular
 III. Whether two push-down automata accept the same language
 IV. Whether a given grammar is context-free

- A. I and II
 B. I and IV
 C. II and III
 D. III and IV

gatecse-2008 theory-of-computation decidability easy

Answer key

6.5.18 Decidability: GATE CSE 2012 | Question: 24

Which of the following problems are decidable?

- Does a given program ever produce an output?
- If L is a context-free language, then, is $\bar{L}$ also context-free?
- If L is a regular language, then, is $\bar{L}$ also regular?
- If L is a recursive language, then, is $\bar{L}$ also recursive?

- A. 1,2,3,4
 B. 1,2
 C. 2,3,4
 D. 3,4

Answer key

6.5.19 Decidability: GATE CSE 2013 | Question: 41



Which of the following is/are undecidable?

1. G is a CFG. Is $L(G) = \phi$?
2. G is a CFG. Is $L(G) = \Sigma^*$?
3. M is a Turing machine. Is $L(M)$ regular?
4. A is a DFA and N is an NFA. Is $L(A) = L(N)$?

A. 3 only

B. 3 and 4 only

C. 1, 2 and 3 only

D. 2 and 3 only

Answer key

6.5.20 Decidability: GATE CSE 2014 | Set 3 | Question: 35



Which one of the following problems is undecidable?

- A. Deciding if a given context-free grammar is ambiguous.
- B. Deciding if a given string is generated by a given context-free grammar.
- C. Deciding if the language generated by a given context-free grammar is empty.
- D. Deciding if the language generated by a given context-free grammar is finite.

Answer key

6.5.21 Decidability: GATE CSE 2015 | Set 2 | Question: 21



Consider the following statements.

- I. The complement of every Turing decidable language is Turing decidable
- II. There exists some language which is in NP but is not Turing decidable
- III. If L is a language in NP, L is Turing decidable

Which of the above statements is/are true?

A. Only II

B. Only III

C. Only I and II

D. Only I and III

Answer key

6.5.22 Decidability: GATE CSE 2015 | Set 3 | Question: 53



Language L_1 is polynomial time reducible to language L_2 . Language L_3 is polynomial time reducible to language L_2 , which in turn polynomial time reducible to language L_4 . Which of the following is/are true?

- I. if $L_4 \in P$, then $L_2 \in P$
- II. if $L_1 \in P$ or $L_3 \in P$, then $L_2 \in P$
- III. $L_1 \in P$, if and only if $L_3 \in P$
- IV. if $L_4 \in P$, then $L_3 \in P$

A. II only

B. III only

C. I and IV only

D. I only

Answer key

6.5.23 Decidability: GATE CSE 2016 | Set 1 | Question: 17



Which of the following decision problems are undecidable?

- I. Given NFAs N_1 and N_2 , is $L(N_1) \cap L(N_2) = \Phi$?
- II. Given a CFG $G = (N, \Sigma, P, S)$ and a string $x \in \Sigma^*$, does $x \in L(G)$?
- III. Given CFGs G_1 and G_2 , is $L(G_1) = L(G_2)$?
- IV. Given a TM M , is $L(M) = \Phi$?

- A. I and IV only B. II and III only C. III and IV only D. II and IV only

gatecse-2016-set1 theory-of-computation decidability easy

Answer key

6.5.24 Decidability: GATE CSE 2017 | Set 1 | Question: 39



Let A and B be finite alphabets and let $\#$ be a symbol outside both A and B . Let f be a total function from A^* to B^* . We say f is *computable* if there exists a Turing machine M which given an input $x \in A^*$, always halts with $f(x)$ on its tape. Let L_f denote the language $\{x\#f(x) \mid x \in A^*\}$. Which of the following statements is true:

- A. f is computable if and only if L_f is recursive.
- B. f is computable if and only if L_f is recursively enumerable.
- C. If f is computable then L_f is recursive, but not conversely.
- D. If f is computable then L_f is recursively enumerable, but not conversely.

gatecse-2017-set1 theory-of-computation decidability difficult difficult

Answer key

6.5.25 Decidability: GATE CSE 2017 | Set 2 | Question: 41



Let $L(R)$ be the language represented by regular expression R . Let $L(G)$ be the language generated by a context free grammar G . Let $L(M)$ be the language accepted by a Turing machine M . Which of the following decision problems are undecidable?

- I. Given a regular expression R and a string w , is $w \in L(R)$?
- II. Given a context-free grammar G , is $L(G) = \emptyset$?
- III. Given a context-free grammar G , is $L(G) = \Sigma^*$ for some alphabet Σ ?
- IV. Given a Turing machine M and a string w , is $w \in L(M)$?

- A. I and IV only B. II and III only C. II, III and IV only D. III and IV only

gatecse-2017-set2 theory-of-computation decidability

Answer key

6.5.26 Decidability: GATE CSE 2018 | Question: 36



Consider the following problems. $L(G)$ denotes the language generated by a grammar G . $L(M)$ denotes the language accepted by a machine M .

- I. For an unrestricted grammar G and a string w , whether $w \in L(G)$
- II. Given a Turing machine M , whether $L(M)$ is regular
- III. Given two grammar G_1 and G_2 , whether $L(G_1) = L(G_2)$
- IV. Given an NFA N , whether there is a deterministic PDA P such that N and P accept the same language

Which one of the following statement is correct?

- A. Only I and II are undecidable B. Only II is undecidable
C. Only II and IV are undecidable D. Only I, II and III are undecidable

gatecse-2018 theory-of-computation decidability easy two-marks

Answer key

6.5.27 Decidability: GATE CSE 2020 | Question: 26



Which of the following languages are undecidable? Note that $\langle M \rangle$ indicates encoding of the Turing machine M .

- $L_1 = \{ \langle M \rangle \mid L(M) = \emptyset \}$
- $L_2 = \{ \langle M, w, q \rangle \mid M \text{ on input } w \text{ reaches state } q \text{ in exactly 100 steps} \}$
- $L_3 = \{ \langle M \rangle \mid L(M) \text{ is not recursive} \}$
- $L_4 = \{ \langle M \rangle \mid L(M) \text{ contains at least 21 members} \}$

A. L_1 , L_3 , and L_4 only

B. L_1 and L_3 only

C. L_2 and L_3 only

D. L_2 , L_3 , and L_4 only

gatecse-2020

theory-of-computation

decidability

two-marks

Answer key

6.5.28 Decidability: GATE CSE 2021 | Set 2 | Question: 36



Consider the following two statements about regular languages:

- S_1 : Every infinite regular language contains an undecidable language as a subset.
- S_2 : Every finite language is regular.

Which one of the following choices is correct?

A. Only S_1 is true

B. Only S_2 is true

C. Both S_1 and S_2 are true

D. Neither S_1 nor S_2 is true

gatecse-2021-set2

theory-of-computation

regular-language

decidability

two-marks

Answer key

6.5.29 Decidability: GATE CSE 2022 | Question: 36



Which of the following is/are undecidable?

- A. Given two Turing machines M_1 and M_2 , decide if $L(M_1) = L(M_2)$.
- B. Given a Turing machine M , decide if $L(M)$ is regular.
- C. Given a Turing machine M , decide if M accepts all strings.
- D. Given a Turing machine M , decide if M takes more than 1073 steps on every string.

gatecse-2022

theory-of-computation

turing-machine

decidability

multiple-selects

two-marks

Answer key

6.5.30 Decidability: GATE CSE 2025 | Set 2 | Question: 15



Let G_1, G_2 be Context Free Grammars (CFGs) and R be a regular expression. For a grammar G , let $L(G)$ denote the language generated by G .

Which ONE among the following questions is decidable?

A. Is $L(G_1) = L(G_2)$?

B. Is $L(G_1) \cap L(G_2) = \emptyset$?

C. Is $L(G_1) = L(R)$?

D. Is $L(G_1) = \emptyset$?

gatecse2025-set2

theory-of-computation

decidability

easy

one-mark

Answer key

6.6

Dpda (1)

6.6.1 Dpda: GATE CSE 2025 | Set 2 | Question: 14



Which ONE of the following languages is accepted by a deterministic pushdown automaton?

- A. Any regular language
- B. Any context-free language
- C. Any language accepted by a non-deterministic pushdown automaton

D. Any decidable language

gatecse2025-set2 theory-of-computation dpda one-mark easy

Answer key

6.7 Finite Automata (43)

Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (15Q) Test 4 (8Q) Weekly Quiz 1 (15Q)

6.7.1 Finite Automata: GATE CSE 1988 | Question: 15

Consider the DFA M and NFA M_2 as defined below. Let the language accepted by machine M be L . What language machine M_2 accepts, if

- $F^2 = A?$
- $F^2 = B?$
- $F^2 = C?$
- $F^2 = D?$

- $M = (Q, \Sigma, \delta, q_0, F)$
- $M_2 = (Q_2, \Sigma, \delta_2, q_{00}, F_2)$

Where,

$$Q_2 = (Q \times Q \times Q) \cup \{q_{00}\}$$
$$\delta_2(q_{00}, \epsilon) = \{\langle q_0, q, q \rangle \mid q \in Q\}$$
$$\delta_2(\langle p, q, r \rangle, \sigma) = \langle \delta(p, \sigma), \delta(q, \sigma), r \rangle$$

for all $p, q, r \in Q$ and $\sigma \in \Sigma$

$$A = \{\langle p, q, r \rangle \mid p \in F; q, r \in Q\}$$
$$B = \{\langle p, q, r \rangle \mid q \in F; p, r \in Q\}$$
$$C = \{\langle p, q, r \rangle \mid p, q, r \in Q; \exists s \in \Sigma^*, \delta(p, s) \in F\}$$
$$D = \{\langle p, q, r \rangle \mid p, q \in Q; r \in F\}$$

gate1988 descriptive theory-of-computation finite-automata difficult

Answer key

6.7.2 Finite Automata: GATE CSE 1991 | Question: 17,b

Let L be the language of all binary strings in which the third symbol from the right is a 1. Give a non-deterministic finite automaton that recognizes L . How many states does the minimized equivalent deterministic finite automaton have? Justify your answer briefly?

gate1991 theory-of-computation finite-automata normal descriptive

Answer key

6.7.3 Finite Automata: GATE CSE 1993 | Question: 27

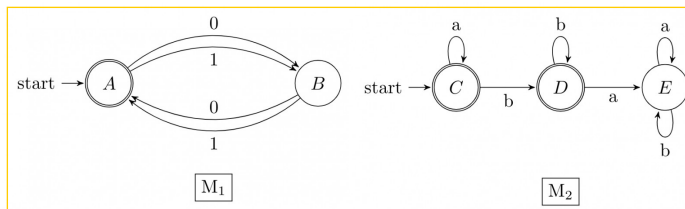
Draw the state transition of a deterministic finite state automaton which accepts all strings from the alphabet $\{a, b\}$, such that no string has 3 consecutive occurrences of the letter b .

gate1993 theory-of-computation finite-automata easy descriptive

Answer key

6.7.4 Finite Automata: GATE CSE 1996 | Question: 12

Given below are the transition diagrams for two finite state machines M_1 and M_2 recognizing languages L_1 and L_2 respectively.



- Display the transition diagram for a machine that recognizes $L_1 \cdot L_2$, obtained from transition diagrams for M_1 and M_2 by adding only ϵ transitions and no new states.
- Modify the transition diagram obtained in part (a) obtain a transition diagram for a machine that recognizes $(L_1 \cdot L_2)^*$ by adding only ϵ transitions and no new states. (Final states are enclosed in double circles).

gate1996 theory-of-computation finite-automata normal descriptive

Answer key

6.7.5 Finite Automata: GATE CSE 1997 | Question: 21

Given that L is a language accepted by a finite state machine, show that L^P and L^R are also accepted by some finite state machines, where

$$L^P = \{s \mid ss' \in L \text{ some string } s'\}$$

$$L^R = \{s \mid s \text{ obtained by reversing some string in } L\}$$

gate1997 theory-of-computation finite-automata proof

Answer key

6.7.6 Finite Automata: GATE CSE 1998 | Question: 1.10

Which of the following set can be recognized by a Deterministic Finite state Automaton?

- The numbers $1, 2, 4, 8, \dots, 2^n, \dots$ written in binary
- The numbers $1, 2, 4, 8, \dots, 2^n, \dots$ written in unary
- The set of binary string in which the number of zeros is the same as the number of ones.
- The set $\{1, 101, 11011, 1110111, \dots\}$

gate1998 theory-of-computation finite-automata normal

Answer key

6.7.7 Finite Automata: GATE CSE 2001 | Question: 5

Construct DFA's for the following languages:

- $L = \{w \mid w \in \{a, b\}^*, w \text{ has baab as a substring}\}$
- $L = \{w \mid w \in \{a, b\}^*, w \text{ has an odd number of a's and an odd number of b's}\}$

gatecse-2001 theory-of-computation easy descriptive finite-automata normal

Answer key

6.7.8 Finite Automata: GATE CSE 2002 | Question: 2.5

The finite state machine described by the following state diagram with A as starting state, where an arc label is x/y , and x stands for 1-bit input and y stands for 2-bit output